

同伦 类型论

数学泛等基础

泛等基础纲领
普林斯顿高等研究院

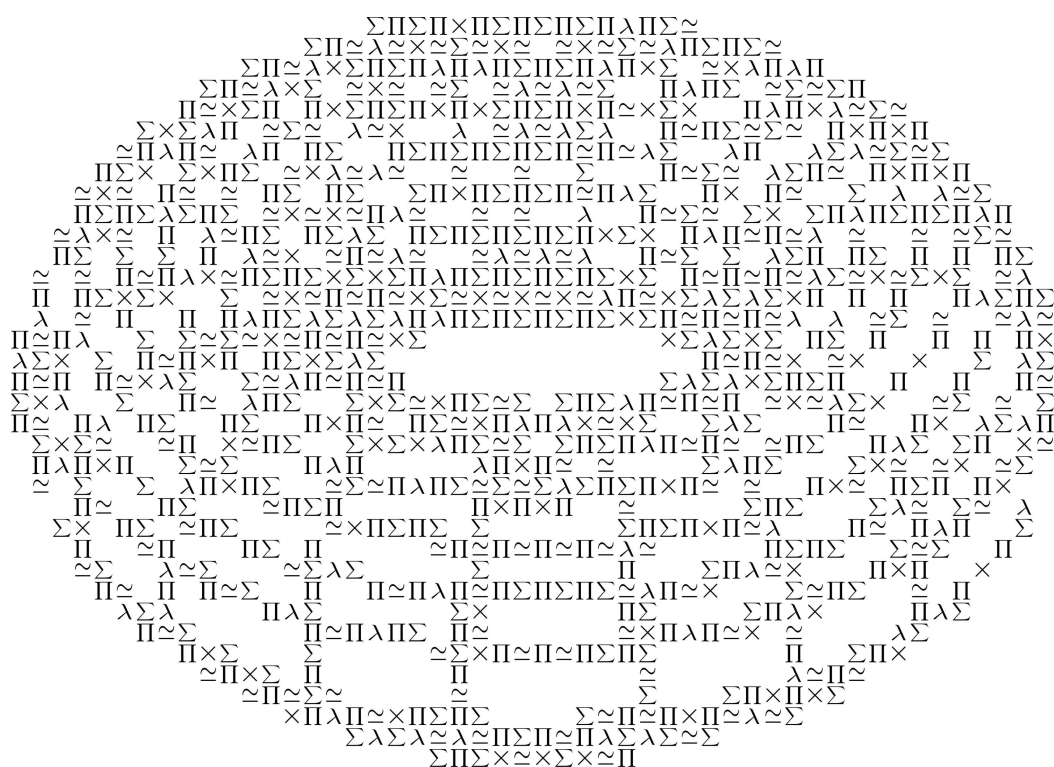
同伦类型论

数学泛等基础

同伦类型论

数学泛等基础

泛等基础纲领
普林斯顿高等研究院



“同伦类型论:数学泛等基础”

© 2013 泛等基础纲领

版本: c201245

MSC 2010 分类号: 03-02, 55-02, 03B15

本作品采用的许可协议为 ***Creative Commons Attribution-ShareAlike 3.0 Unported License***. 协议副本见 <http://creativecommons.org/licenses/by-sa/3.0/>.

本书可在 <http://homotopytypetheory.org/book/>. 中免费获得

致谢

除了普林斯顿高等研究所的慷慨支持, 本书的部分撰稿人还得到了下列机构和基金的部分或全部资助

- 普林斯顿高等研究所成员协会: 向普林斯顿高等研究院提供资助
- 斯洛文尼亚研究署: P1-0294, N1-0011.
- 美国空军科学研究办公室: FA9550-11-1-0143, and FA9550-12-1-0370.

本材料部分基于上述奖项下由美国空军科学研究办公室资助的工作。本出版物中表达的任何观点、发现、结论或建议均属于作者个人意见, 并不一定反映美国空军科学研究办公室的立场。

- 英国工程与物理科学研究委员会: EP/G034109/1, EP/G03298X/1.
- 由欧盟第七框架计划资助(资助协议号 243847) (ForMath).
- 美国国家科学基金会: DMS-1001191, DMS-1100938, CCF-1116703, 与 DMS-1128155.

本研究成果部分基于美国国家科学基金会在上述奖项下的资助。作者在本材料中阐述的任何意见、发现、结论或建议均不代表美国国家科学基金会的观点。

- 西蒙尼基金:向普林斯顿高等研究院提供资助

前言

普林斯顿高等研究院“泛等基础”主题年

普林斯顿高等研究院“泛等基础”主题年于2012-13年在普林斯顿高等研究所数学院举行。由Steve Awodey, Thierry Coquand与Vladimir Voevodsky组织。

以下为官方参与者:

Peter Aczel	Eric Finster	Alvaro Pelayo
Benedikt Ahrens	Daniel Grayson	Andrew Polonsky
Thorsten Altenkirch	Hugo Herbelin	Michael Shulman
Steve Awodey	André Joyal	Matthieu Sozeau
Bruno Barras	Dan Licata	Bas Spitters
Andrej Bauer	Peter Lumsdaine	Benno van den Berg
Yves Bertot	Assia Mahboubi	Vladimir Voevodsky
Marc Bezem	Per Martin-Löf	Michael Warren
Thierry Coquand	Sergey Melikhov	Noam Zeilberger

此外, 还有以下学生, 他们的参与同样宝贵

Carlo Angiuli	Guillaume Brunerie	Egbert Rijke
Anthony Bordg	Chris Kapulkin	Kristina Sojakova

此外, 下列短期与长期来访者(包括学生来访者)对主题年的贡献亦不可或缺。

Jeremy Avigad	Richard Garner	Nuo Li
Cyril Cohen	Georges Gonthier	Zhaohui Luo
Robert Constable	Thomas Hales	Michael Nahas
Pierre-Louis Curien	Robert Harper	Erik Palmgren
Peter Dybjer	Martin Hofmann	Emily Riehl
Martín Escardó	Pieter Hofstra	Dana Scott
Kuen-Bang Hou	Joachim Kock	Philip Scott
Nicola Gambino	Nicolai Kraus	Sergei Soloviev

关于此书

写书并非我们的本意。现在的工作源于我们的一次集体尝试。我们试图构造一种能够被人类所理解的新型“非形式化类型论”, 作为能被机器检查的形式化类型论的补充。泛等基础与一种

能被计算机证明助手所实现的数学基础关系密切。尽管形式化并非本书的一部分, 此处呈现的许多材料都是先在一个证明助手中完全形式化后再“非形式化”为你所看见的表述——这与形式化数学的通常做法明显相反。

上述每一个人都以思想、言辞或行动的方式对主题年与本书作出了贡献。贯穿全年的合作精神着实非同凡响。

特别感谢普林斯顿高等研究所。没有其协助, 这本书永远不可能出现。事实证明, 这里是创造新数学的理想环境: 激励人心, 氛围融洽且支持性强希望这样一种氛围的踪迹能萦绕在这本书的书页, 与这一个新领域的进一步的发展中。

泛等基础纲领
普林斯顿高等研究院
普林斯顿, 2013年四月

目录

序	1
第一部分 基础	11
第1章 类型论	13
1.1 类型论 vs 集合论	13
1.2 函数类型	16
1.3 宇宙与族	18
1.4 依值函数类型 (Π -类型)	18
1.5 乘积类型	20
1.6 依值对类型 (Σ -类型)	23
1.7 余积类型	25
1.8 布尔类型	26
1.9 自然数	28
1.10 模式匹配与递归	30
1.11 命题即类型	31
1.12 恒等类型	36
备注	41
练习	43
第2章 同伦类型论	45
2.1 Types are higher groupoids	48
2.2 Functions are functors	56
2.3 Type families are fibrations	57
2.4 Homotopies and equivalences	60
2.5 The higher groupoid structure of type formers	64
2.6 Cartesian product types	65
2.7 Σ -types	68
2.8 The unit type	70
2.9 Π -types and the function extensionality axiom	70
2.10 Universes and the univalence axiom	73
2.11 Identity type	74
2.12 Coproducts	76
2.13 Natural numbers	79
2.14 Example: equality of structures	80

2.15 Universal properties	83
备注	85
练习	87
第3章 集合与逻辑	89
3.1 Sets and n -types	89
3.2 Propositions as types?	91
3.3 Mere propositions	93
3.4 Classical vs. intuitionistic logic	94
3.5 Subsets and propositional resizing	96
3.6 The logic of mere propositions	98
3.7 Propositional truncation	98
3.8 The axiom of choice	100
3.9 The principle of unique choice	102
3.10 When are propositions truncated?	102
3.11 Contractibility	104
备注	107
练习	108
第4章 等价	111
4.1 Quasi-inverses	112
4.2 Half adjoint equivalences	114
4.3 Bi-invertible maps	117
4.4 Contractible fibers	118
4.5 On the definition of equivalences	119
4.6 Surjections and embeddings	119
4.7 Closure properties of equivalences	121
4.8 The object classifier	123
4.9 Univalence implies function extensionality	125
备注	127
练习	127
第5章 归纳法	129
5.1 Introduction to inductive types	129
5.2 Uniqueness of inductive types	131
5.3 W-types	134
5.4 Inductive types are initial algebras	137
5.5 Homotopy-inductive types	139
5.6 The general syntax of inductive definitions	143
5.7 Generalizations of inductive types	146
5.8 Identity types and identity systems	149
备注	153
练习	153

第6章 高阶归纳类型	157
6.1 Introduction	157
6.2 Induction principles and dependent paths	159
6.3 The interval	163
6.4 Circles and spheres	164
6.5 Suspensions	166
6.6 Cell complexes	169
6.7 Hubs and spokes	170
6.8 Pushouts	171
6.9 Truncations	175
6.10 Quotients	177
6.11 Algebra	182
6.12 The flattening lemma	186
6.13 The general syntax of higher inductive definitions	191
备注	193
练习	194
第7章 同伦 n-类型	197
7.1 Definition of n -types	197
7.2 Uniqueness of identity proofs and Hedberg's theorem	200
7.3 Truncations	204
7.4 Colimits of n -types	209
7.5 Connectedness	213
7.6 Orthogonal factorization	217
7.7 Modalities	222
备注	225
练习	226
第二部分 数学	231
第8章 同伦论	233
8.1 $\pi_1(S^1)$	236
8.2 Connectedness of suspensions	244
8.3 $\pi_{k \leq n}$ of an n -connected space and $\pi_{k < n}(S^n)$	245
8.4 Fiber sequences and the long exact sequence	246
8.5 The Hopf fibration	250
8.6 The Freudenthal suspension theorem	255
8.7 The van Kampen theorem	261
8.8 Whitehead's theorem and Whitehead's principle	269
8.9 A general statement of the encode-decode method	272
8.10 Additional Results	274
备注	274
练习	276

第9章 范畴论	277
9.1 Categories and precategories	278
9.2 Functors and transformations	281
9.3 Adjunctions	284
9.4 Equivalences	285
9.5 The Yoneda lemma	290
9.6 Strict categories	293
9.7 $\dagger$ -categories	294
9.8 The structure identity principle	295
9.9 The Rezk completion	298
备注	304
练习	305
第10章 集合论	307
10.1 The category of sets	307
10.2 Cardinal numbers	315
10.3 Ordinal numbers	318
10.4 Classical well-orderings	324
10.5 The cumulative hierarchy	327
备注	332
练习	332
第11章 实数	337
11.1 The field of rational numbers	338
11.2 Dedekind reals	338
11.3 Cauchy reals	344
11.4 Comparison of Cauchy and Dedekind reals	361
11.5 Compactness of the interval	362
11.6 The surreal numbers	369
备注	379
练习	380
Appendix	383
附录 A 形式化类型论	385
A.1 The first presentation	387
A.2 The second presentation	391
A.3 Homotopy type theory	397
A.4 Basic metatheory	398
备注	400

参考文献	401
------	-----

Index of symbols	411
------------------	-----

Index	417
-------	-----

序

同伦类型论是一个全新的数学分支, 其以出人意料的方式结合了多个不同的领域。它基于近期发现的同伦论和类型论之间的联系。同伦论是代数拓扑与同调代数的产物, 与高阶范畴论有着联系; 而类型论则是数理逻辑和理论计算机科学的一个分支。尽管二者之间的联系仍是研究的重点, 但愈发清晰的是, 这只是一个需要更多时间和努力来理解的学科的开端。它还会涉及到像球面的同伦群, 类型检查算法, 弱 ∞ -群胚的定义这样看起来十分遥远的话题。

同伦类型论也为数学基础带来了新思想。一方面, 我们有Voevodsky提出的精美的泛等公理由泛等公理, 我们可以得到, 同构的结构实际上可以是恒等的, 这是数学家在工作时一直乐于使用的一个原则, 即使其与传统的“官方”的数学基础并不兼容。另一方面, 我们有高阶归纳类型, 其提供了对同伦论中基本空间和构造的直接、逻辑化的描述: 如球面, 柱体, 截断, 局部化, 等等。这些思想在经典的集合论的基础中是不可能被直接捕捉到的。但当它们在同伦类型论中结合在一起时, 一种全新的“同伦类型的逻辑”浮现了出来。

这表明了对数学基础的一种新的洞见, 其具有内在的, 直觉化的同伦论的内容——即数学对象的“不变量”的概念——与便捷的机器实现。后者可以为在职数学家提供实用的帮助。这就是泛等基础纲领。本书被设计为对泛等基础的基本概念进行表述的第一个系统性文献。同时也是通过这种新方式进行推理的例子的合集——但是不需要读者了解或学习任何形式逻辑, 或是使用任何的证明助手。

需要强调的是, 同伦类型论还是一个新兴的领域, 而泛等基础也是一个还在进行中的工作。这本书应被视作这个对这个领域的一部分, 在写作时拍摄的一张“快照”, 而不是经过润色的, 对一个完整学说进行的阐释。就像我们稍后会大致讨论的, 同伦类型论的许多方面都还未被充分地理解——而还有一些甚至不会在此处被提及。几乎可以肯定的是, 尽管最终的理论和本书所描述的理论不会完全相同, 但其至少会和这个理论一样有力; 因此, 我们相信泛等基础最终将成为集合论的一个切实可行的替代, 作为大多数数学家进行的非形式化数学研究的“隐形基础”。

类型论

类型论最初由Bertrand Russell[Rus08], 发明, 作为一种遏制当时在数学基础研究中出现的悖论的手段。在接下来的几十年里, 它被许多人进一步发展, 其中Church [Chu40, Chu41]将他的 λ -演算与类型论相结合。尽管其还未被广泛视作经典数学的基础——集合论在此更加流行——类型论仍然由众多的应用, 特别是在计算机科学与编程语言理论 [Pie02]。

Per Martin-Löf [ML98, ML75, ML82, ML84], 与其他人一起构造了对Church的类型论修改后的一种“谓词性”版本。现在通常称其为一个依值的, 构造性的, 直觉主义的类型论, 或简称为Martin-Löf类型论。这是我们在本书中所考虑的系统的基礎; 其一开始被视作对构造性数学进行形式化的严谨框架。之后, 我们通常使用“类型论”来特别指代这一系统、或与其相似的系统, 尽管类型论这个学科涵盖的概念更加广泛。(见[Som10, KLN04]以了解类型论的历史)

类型论与集合论不同的不同之处在于, 我们使用类型这一基础概念来分类对象, 就像编程语言中所使用的数据类型一样。这些精巧的结构化的类型被用于表示被分类对象的具体规范, 并提供对这些对象进行推理的规则。举一个简单的例子, 众所周知, 具有积类型 $A \times B$ 的对象的形式是 (a, b) , 因此人们自然知道该如何构造和分解它们。类似地, 通过一个使用类型为 A 的对象编码

的, 具有类型 B 的对象, 我们可以得到具有函数类型 $A \rightarrow B$ 的对象。并且给定一个类型为 A 的参数, 其值可以被计算。这种所有对象均有的, 坚实可预测的行为(与集合论中更加自由的形成规则相对, 其允许非齐次集的存在), 使类型论在检验计算机程序正确性方面得以广泛应用。而其清晰的推理规则, 而与类型的构造相结合, 形成了现代证明助手的基础, 其在形式化数学与检验形式化证明正确性中被应用。我们将在下文继续探讨类型论的这一方面。

然而, 从数学角度理解类型论始终存在一个难题, 类型论的基本概念类型与集合概念在本质上的差异一直难以被精确刻画。我们相信, 将类型不再视为某种奇异集合(或许无需经典逻辑构造而成), 而是从同伦论视角将其理解为空间, 这一新思路是一次重大突破。尤其值得关注的是, 它解决了长久以来的困惑: 为何类型中元素的相等性概念会与集合中元素的相等性存在根本差异。

在同伦论中, 人们关注的是空间与它们之间在同伦意义上的连续映射。在一对连续映射 $f : X \rightarrow Y$ 与 $g : X \rightarrow Y$ 之间, 如果一个连续映射 $H : X \times [0, 1] \rightarrow Y$ 满足 $H(x, 0) = f(x)$ 和 $H(x, 1) = g(x)$, 我们就称其为同伦。同伦 H 可以被视作一种从 f 到 g 的“连续形变”。如果存在一堆方向相反的连续映射, 其复合同伦于单位映射, 我们就说空间 X 和 Y 是同伦等价的 $X \simeq Y$, 即, 它们在“同伦意义上”是同构的。同伦等价的空间具有相同的代数不变量(例如, 同调群或基本群), 因此称它们具有相同的同伦类型。

同伦类型论

同伦类型论(HoTT)从同伦论的视角解读类型论。同伦类型论中, 我们将类型视作“空间”或(像同伦论已研究的那样)视作高阶群胚。而逻辑构造(就像积 $A \times B$)则被视作这些空间之间的同伦不变量的构造。这样一来, 我们得以直接操纵空间, 而无需首先构造点集拓扑(或任何它的组合替代理论, 例如单纯集论)为了简要地介绍这种视角, 我们先考虑类型论的基础概念, 或确切地说: 项 a 具有类型 A , 写作

$$a : A$$

传统上, 这样一个表达式被认为类似于

“ a 是集合 A 的元素”

然而, 在同伦类型论中, 我们认为其类似于

“ a 是空间 A 中一点”

类似地, 类型论中每个函数 $f : A \rightarrow B$ 都被视作从空间 A 到空间 B 的一个连续映射。需要强调的是, 这些“空间”都是在纯粹同伦论意义上对待的, 而不具有拓扑意义。例如, 不存在类型的“开子集”的概念或是类型中元素列的“收敛性”。我们只有“同伦的”概念, 例如点之间的道路与道路之间的同伦, 这些概念在同伦论的其它模型下也依然有意义(例如单纯集)因此, 把类型视作 ∞ -群胚是更加准确的说法。这是同伦论中“不变量对象”的名称, 可以用拓扑空间, 单纯集, 或其他任何同伦论的模型表示。然而, 偶尔使用拓扑学中“空间”和“道路”这样的名称是很方便的, 只要我们记得其他的拓扑概念是不可用的就好。

(为这些对象使用同伦类型这个短语也是很诱人的, 这暗示了一对对偶的解读: “在同伦意义上被看待的(类型论的)类型”和“从同伦论的观点上看待空间”。后者与经典意义上的“同伦类型”有些许不同——空间同伦等价意义下的等价类, 尽管其确实保持了像“这两个空间具有相同的同伦类型”这样的短语的含义)

将类型解读为结构化对象而非集合的想法有着悠长的历史, 众所周知, 它可以阐明类型论中许多神秘的方面。例如, 将类型解读为层能够帮助解释类型论的直觉主义本质, 而将其解读为偏等价关系或“定义域”则有助于解释它的可计算性。这也暗示着我们可以使用类型论的推理来研究结构化对象, 这指向了范畴论逻辑这一丰富的领域。对其的同伦论解读也遵循了同样的模式: 它阐明了类型论中恒等(或相等性)的本质, 允许我们在同伦论的研究中使用类型论式的推理。

同伦论解读的核心观点为: 对具有同样类型 A 的两个对象 $a, b : A$, 逻辑上的恒等的概念 $a = b$ 可以被理解为在空间 A 中存在一条从点 a 到 b 的 $dp : a \leadsto b$ 。这也意味着如果两个函数 $f, g : A \rightarrow B$ 是同伦的, 它们就是恒等的。因为同伦只不过是 B 上的一个(连续)道路族 $p_x : f(x) \leadsto g(x)$, 其中 $x : A$ 。

在类型论中, 对每个类型 A , 都有一个(在先前些许神秘的)类型 Id_A , 它是 A 上两个物体的等同证明的类型。在同伦类型论中, 这只不过是从单位区间到空间 A 的全部连续映射 $I \rightarrow A$ 所构成的道路空间 A^I 在这个意义上, 项 $p : \text{Id}_A(a, b)$ 呈现了 A 上的一条道路 $p : a \leadsto b$ 。

同伦类型论的思想在2006年左右, 在Awodey与Warren的合著 [AW09], 和Voevodsky的著作 [Voe06]中分别被提出, 但其灵感源于Hofmann与Streicher的早期群胚解读 [HS98]。实际上, 高阶范畴论(特别是弱 ∞ -群胚理论)与同伦论之间的亲密联系是众所周知的事。这一观点最先由Grothendieck提出, 现在同时被两个领域的数学家所关注。Awodey–Warren和Voevodsky原先所提出的语义模型使用了同伦论中所熟知的概念与技巧。这些概念与技巧现在也在高阶范畴论中被使用, 像是Quillen模型范畴和Kan单纯集。

实际上, Voevodsky构造了类型论的Kan单纯集解读, 并意识到这种解读满足一个更深刻的性质, 他称其为泛等。这一性质先前并未在类型论中被考虑过(尽管Church的命题外延性原理被发现是该性质的一个特例, 而Hofmann和Streicher之前考虑过另一个称为“宇宙外延性”的特例)。将泛等以一种新公理的方式添加到类型论中有着深远的结果, 其中的许多是很自然, 简单且鼓舞人心的。泛等公理也增强了同伦视角下的类型论, 因为它在单纯模型和其他相关模型中成立, 而在将类型视作集合的观点下不成立。

泛等基础

简略地说, 泛等公理的基本思想可以解释如下: 在类型论中, 人们可以有一个元素本身为类型的一种类型; 这样一种类型叫做宇宙并且通常记作 $\mathcal{U}$ 。类型为 $\mathcal{U}$ 的对象的类型通常又称为小类型。像其他类型一样, $\mathcal{U}$ 有着恒等类型 $\text{Id}_{\mathcal{U}}$, 其表达了小类型间的恒等关系 $A = B$ 。将类型视作空间, $\mathcal{U}$ 是一个点为空间的 $\mathcal{U}$; 为了理解其恒等类型, 我们必须知道, $\mathcal{U}$ 上空间 A 与 B 之间的道路 $p : A \leadsto B$ 是什么? 泛等公理告诉我们, 这样的一条道路对应于同伦等价 $A \simeq B$, (大体上)就像前面所解释的那样。更精确地说, 给定任意(小)类型 A 与 B , 除了 A 与 B 之间的等同证明的基础类型 $\text{Id}_{\mathcal{U}}(A, B)$, 还有定义类型 $\text{Equiv}(A, B)$, 其对象是从 A 到 B 的等价。因为任何物体上的恒等映射都是一个等价, 因此有一个典范映射

$$\text{Id}_{\mathcal{U}}(A, B) \rightarrow \text{Equiv}(A, B).$$

泛等公理声称这样一个映射其本身也是一个等价。尽管有着过度简化的风险, 我们可以将其简洁地描述如下:

泛等公理: $(A = B) \simeq (A \simeq B)$.

换言之, 恒等与等价等价。特别地, 人们会说“等价的类型恒等”。然而, 这样一个短语有些误导人, 因为这种说法听起来像是把等价的观念坍缩至与恒等的概念重合, 像一种的“骨骼性”条件。而实际上, 泛等公理是把恒等的概念扩张至与(未变的)等价的观念重合。从同伦论的观点上看, 泛等蕴含着具有相同同伦类型的空间被宇宙 $\mathcal{U}$ 上的一条道路所连接, 这与(小)空间的分类空间的直观理解相符。从逻辑性的观点上看, 这却是一个爆炸性的新思想: 它声称同构的事物可以是恒等的! 数学家在实践中总是理所当然的等同同构的事物, 但他们通常通过“滥用概念”, 或是通过其它非形式化的方法来这么做, 认为这些对象“实际上”并不恒等。然而在这个新基础的框架中, 这样的结构可以形式化为逻辑学上的恒等, 即每个涉及了其中一者的性质或构造可以应用于另一者。实际上, 这样的等同证明现在可以被显式地作出, 而那些性质和构造可以沿着该证明被系统性地传递给另外一者。不仅如此, 等同证明的不同形式使得它们自身形成了一种可以(且应当!) 考虑的结构。

总之, 对宇宙 $\mathcal{U}$ 中的两点 A 与 B (即小类型), 泛等公理将以下三个概念等同:

- (逻辑学) A 与 B 间的等同证明 $p : A = B$
- (拓扑学) $\mathcal{U}$ 上从 A 到 B 的一条道路 $p : A \leadsto B$
- (同伦论) A 与 B 间的一个等价 $p : A \simeq B$

高阶归纳类型

类型论的一个经典优势是, 在研究归纳定义的结构时, 其有着简单而有效的技巧。最简单的非平凡归纳定义的结构是自然数, 由零与后继函数归纳生成。从这句话中, 人们可以把数学归纳法抽象为一个算法, 其刻画了自然数的特征。更一般的归纳定义包括各种类型的链表与良基树, 它们的特征都被一个对应的“归纳原理”所刻画。这包括了某些编程语言中使用的大多数数据结构; 正因如此, 类型论在对后者进行形式化推理方面才如此实用。如果从非常广义的角度来理解, 归纳定义同样包含诸如无交并 $A + B$ 这样的例子。其可以看作是由两个投射 $A \rightarrow A + B$ 与 $B \rightarrow A + B$ “归纳”生成的。这种情况下, 其“归纳原理”即为“分类讨论”, 其刻画了无交并的特征。

在同伦论中, 考虑“归纳定义空间”是很自然的想法。其不仅可以由一族点生成的, 还可以由一族道路和高阶道路所生成。在经典理论中, 这种空间被称为 CW复形。例如, 圆周 S^1 由单个点和一条从该点出发回到自身的道路生成。类似地, 2-球面 S^2 由单个点 b , 一条从 b 上常道路出发回到自身的二维道路生成。而环面 T^2 则由单个点, 从该点出发回到自身的两条道路 p 与 q , 以及一条从 $p \cdot q$ 到 $q \cdot p$ 的一条二维道路生成。

通过同伦类型论中道路和等同性的联系, 这种“归纳定义空间”可以由“归纳原理”在类型论中被刻画, 与像自然数和无交并这样的经典例子完全相似。这就产生了高阶归纳类型, 对例如球面这样的熟悉的空間, 它给出了一个直接的“逻辑学”的方法来进行推理, (与泛等性结合后)可以用于进行同伦论中熟悉的论证, 像是计算球面的同伦群, 但是以纯粹形式化的方式进行。最终的结果是经典同伦论与经典类型论思想的结合, 进而对双方都带来了新的认知。

然而, 这只是冰山一角: 同伦论中许多抽象的构造, 如同伦余极限, 纬悬, Postnikov塔, 局部化, 完备化, 以及谱化, 也可以被表达为高阶归纳类型。其中的许多在经典理论中都使用Quillen的“小对象论证”来构造。该论证可以视作一个有限算法, 用于描述空間的一个无限CW复形表示。就像“零与后继”是对自然数集这个无限集的有限算法描述一样。由小对象论证产生的空間以其复杂性和难以理解性而臭名昭著; 类型论的方法则更加简单, 通过直接给予合适的“归纳原理”绕过了任何显式的构造。因此, 泛等性和高阶归纳类型的结合暗示了同伦论在实践方面发生变革的可能。

泛等基础中的集合

我们宣称泛等公理最终可以作为“一切”数学的基础, 但截至目前我们所讨论的只有同伦论。当然, 存在很多不使用同伦类型论的新特性, 而应用类型论来形式化数学的具体例子, 例如最近在Coq [GAA⁺13]中对Feit–Thompson奇阶定理的形式化。

但是传统数学是基于集合论的, 也就是所有的数学的对象与构造都可以被编码进像Zermelo–Fraenkel集合论(ZF)这样的理论。然而, 现已公认, 对于除了集合论以外的绝大多数数学领域, ZF中集合错综复杂的分层式的成员关系结构实在没有必要: 像Lawvere的集合范畴基本理论 [Law05]这样的, 更加“结构化”的集合论是足够的。

在泛等基础中, 基础对象是“同伦类型”而非集合, 但是我们可以定义一类行为类似于集合的类型。从同伦论的角度, 这些类型可以看作是每个联通分量均可缩的空間, 即, 同伦等价于离散空間的空間。有一个定理表明, 这样的“集合”构成的范畴满足Lawvere’s公理(或相关的公理, 取决于理论的具体细节)。因此, 任何能够用ETCS-式理论表示的数学理论(经验表明, 这基本上是全部的数学理论)都可以等价地在泛等基础中表示。

这支持了我们所宣称的泛等基础至少和现存的数学基础一样好的观点。使用泛等基础的数学家可以利用集合来构造结构, 就像他们所熟悉的那样。而更一般化的同伦类型则在幕后待命。因

此,在这本书所选择的大部分应用场景中,泛等基础能为此作出不同于现存基础系统的新贡献。

不出意料地,同伦论和范畴论属于这种应用范围。但不太明显的是泛等基础甚至也能为像集合论和实分析这样的学科带来一些新奇有趣的概念。例如,泛等公理允许我们把同构的结构等同,而高阶归纳类型允许我们通过泛性质来直接描述对象。因此我们可以避免使用任意选择的代表元或是超限迭代构造,事实上,甚至是ZF集合论所研究的对象也可以通过这样一种归纳泛性质在泛等基础的集合中被刻画。

非形式化类型论

传统数学家在学习类型论中经常遇到的一个难点是,类型论通常在一个全部或部分形式化的推理系统中呈现。这样一种风格对证明论的研究非常有利,但在应用性的非形式化的推理中不是特别方便。不仅是大多数的在职数学家,即使是对数学基础感兴趣的人都会对此感到陌生。这个作品的一个目标是构建一个在泛等基础下的非形式化的数学研究风格。这种风格不失严谨性与精准性,同时又与日常的数学语言与表达风格相近。

在现在的数学工作中,对数学对象进行构造和推理时,人们总是假定其方法原则上可以在一个例如ZFC的基础集合论上形式化——至少在有足够的智慧和耐心的情况下是这样的。而在绝大多数情况下,人们甚至根本不需要意识到这种可能性,因为这与一个证明是“充分严谨”的条件大致重合。(也就是说所有接受了教育且经验充足的数学家都能直观地理解)但人们确实需要知道小心地对待“非形式化集合论”的少数几个方面:例如对对象总体的使用过于宽泛或不明确以至于无法成为集合;选择公理与其等价形式;甚至于(对本科生而言)反证法;诸如此类。将同伦类型论这样的新的基础系统作为非形式化推理的隐式基础需要调整人们的一些直觉和技巧。本书致力于作为这种“新数学”的样例,虽然依然是非形式化的,但原则上是可在同伦类型论而非ZFC中被形式化的——同样地,只要有着足够的智慧和耐心。

值得强调的是,在这种系统中,这样一种形式化有着切实的好处。类型论的形式化系统与计算机系统相适配,并已在现存的证明助手中被实现。证明助手是一个电脑程序,在完全形式化的证明中指导用户,只允许推理中有效的步骤。它也提供一定程度的自动化,例如从库中搜索现成的定理,并且能够从最终(构造性)的证明中提取出数值算法。

我们相信泛等基础的这一方面将其与其他的基础分别出来,有着为在职数学家提供新的实用工具的潜力。实际上,基于较老的类型论的证明助手已经被用于形式化大量的数学证明,例如四色定理与Feit-Thompson定理。泛等基础的计算机实现目前仍在开发中(就像理论自身一样)。然而,即使目前可用的实现也已经演示了他们的价值(它们基本上是对像Coq和Agda这样已有的证明助手的小小的修改)。其不仅包括对已知证明的形式化,也包括对新证明的发现。实际上,书中描述的许多的证明实际上都是先在一个证明助手中以完全形式化的方法完成的,并在此时才被第一次“非形式化”——这与传统的形式化与非形式化数学的关系相反。

人们可以想象,在不太遥远的未来,可能会发生这种事:数学家们在泛等基础的系统中检验他们自己论文的正确性,将其在一个证明助手中形式化,而且这么做就像是用 $\text{\LaTeX}$ 来撰写他们的论文一样自然原则上,这对其他的基础系统也是同样有可能实现的,但我们相信使用泛等基础是更加切实可行的,这在本书以及其形式化部分可见一斑。

可构造性

经典基础与类型论的最令人惊讶的区别是证明相关性的概念——其主张数学陈述,甚至是它们的证明,是一等数学对象。在类型论中,我们通过类型表示数学陈述,其可以同时被视作数学构造与数学断言,这是一个称之为命题即类型的构想。据此,我们可以将项 $a : A$ 同时视作类型 A 的元素(或是同伦类型论中空间 A 的一点)与对命题 A 的一个证明。举个例子,假设我们有集合 A 与 B (离

散空间), 考虑陈述“ A 同构于 B ”。在类型论中, 其可以描绘为:

$$\text{Iso}(A, B) := \sum_{(f:A \rightarrow B)} \sum_{(g:B \rightarrow A)} \left((\prod_{(x:A)} g(f(x)) = x) \times (\prod_{(y:B)} f(g(y)) = y) \right).$$

在此处, 将类型构造子 $\Sigma, \Pi, \times$ 分别读作“存在”, “对于任意”与“且”, 便可得到“ A 与 B 是同构的”的通常表述。另一方面, 把他们读作和与积则可得 A 与 B 之间全部同构的类型! 为了证明 A 与 B 是同构的, 人们需要构造一个证明 $p : \text{Iso}(A, B)$ 。因此这与构造从 A 到 B 的一个同构是一样的, 即, 展示一对函数 f, g 与它们的复合为两边的恒等映射的证明。反之, 后者只不过是恰当的同伦。在这一点上, 证明一个命题与构造。实际上, 证明一个具有形式“ A 且 B ”的陈述就是证明 A 且证明 B , 即, 给出类型 $A \times B$ 的一个元素。而证明 A 蕴含 B 就是寻找一个类型为 $A \rightarrow B$ 的元素, 即一个从 A 到 B 的函数(决定一个从 A 的证明到 B 的证明的映射)。

命题即类型的逻辑十分灵活, 并且有着非常多的变种, 例如只使用类型的一个子类来描述命题。在同伦类型论中, 这样的子类是自然显现出来的。因为由所有类型组成的系统, 会像经典同伦论中的空间一样, 根据其高阶同伦结构存在或坍塌的维度分层。特别地, Voevodsky找到了一种对同伦 n -类型的纯类型论的定义, 与在 n 维以上维度中不含有非平凡同伦信息的空间对应。(0-类型是前文所提的满足Lawvere公理的“集合”) 不仅如此, 与高阶归纳类型结合在一起, 我们可以普遍地把一个类型“截断”为一个 n -类型; 在经典同伦论中这是其第 n 个Postnikov截面。对逻辑学而言特别重要的是 (-1) -类型, 其被我们称为纯粹命题。在经典理论中, 每个 (-1) -类型要么空要么可缩; 我们将这些情况分别解读为“假”和“真”。

将所有类型都作为命题使用会产生一个十分“构造性”的逻辑构想; 见 [Kol32, TvD88a, TvD88b]以了解更多。举例而言, 每个关于某物存在的证明都会携带充分的信息, 使得该对象真的可以被找到; 对每个“ A 或 B ”成立的证明, 它要么是 A 成立的证明, 要么是 B 成立的证明因此, 我们可以自动地从每个证明中提取出一个算法; 这在应用于计算机编程时非常有用

然而另一方面, 这种逻辑与数学对存在性证明的传统理解存在偏差。事实上, 其并不忠实地遵守一些重要的传统推理原则, 比如选择公理和排中律。对于这些, 我们需要使用“ (-1) -截断”逻辑, 其中只有同伦 (-1) -类型表示命题。

更具体地, 一方面考虑选择公理: “如果对每个 $x : A$, 存在一个 $y : B$ 使得 $R(x, y)$, 则存在一个函数 $f : A \rightarrow B$ 使得对所有的 $x : A$, 有 $R(x, f(x))$ 。”命题即逻辑纯粹的“存在”概念足够强大, 以至于这个陈述可以简单地被证明——尽管其没有通常的选择公理所能产生的全部结果。然而, 在 (-1) -截断逻辑中, 这一陈述并不是自动为真, 而是一个强力的假设, 与在经典集合论中所对应的版本有着一样结果。

另一方面, 考虑排中律: “对所有的 A , 要么 A 要么非 A 。”在纯粹的命题即类型逻辑中解释会产生一个与泛等公理不一致的陈述。因为证明“ A ”意味着给出其中的一个元素, 这个假设将会给出一个从所有非空类型中选择元素的统一方式——一种希尔伯特选择算子。泛等蕴含着 A 中通过该算子选出的元素在所有 A 的自等价下必须不变, 因为这些自等价与自恒等是等同的, 而每个操作都必须保恒等; 但显然有些类型有着无不动点的自同构, 例如我们可以交换只有两个元素的类型的元素。然而, “ (-1) -截断排中律”, 虽然也不是自动为真, 但可被无矛盾地引入, 且其在经典数学中的绝大多数结论依然成立。

换言之, 虽然纯粹的命题即类型逻辑在和前文所提及的强烈的算法意义下是“构造性的”, 但默认的 (-1) -截断逻辑是在另一种意义下的“构造性的”(准确地说, 是海廷形式化的名为“直觉主义”的逻辑) 在后者中, 我们可以自由地添加选择公理和排中律来得到被称为“经典”的逻辑。因此, 同伦类型论和构造性逻辑与经典逻辑的构想都兼容, 也可容纳远多于此的其他逻辑范式。实际上, 同伦论视角揭示了经典逻辑与构造性逻辑是可以并行不悖的, 就像是不同系统构成的光谱的两端, 其中有着无尽的可能性(满足 $-1 < n < \infty$ 的同伦 n -类型)。我们可以讨论带层级标记的“ LEM_n ”与“ AC_n ”: 其中 AC_∞ 是可证明的, 而 LEM_∞ 则与泛等公理不相容; 另一方面, AC_{-1} 与 LEM_{-1} 正是经典数学家所熟悉的版本(因此在不标注下标时, 默认指 (-1) 层级是合适的)。事实上,

我们完全可以构建这样一类有用的逻辑系统: 其中仅有特定类型需要满足这类进阶的“经典”原则, 而所有类型在整体上仍保持“可构造性”。

值得强调的是, 泛等基础并不要求使用构造性的或直觉主义逻辑。大多数依赖于排中律和选择公理的经典数学结果都可以在泛等公理中进行, 只需要简单地假设这两个原理成立(在正确的——(−1)-截断——形式下)。然而, 出于一些原因, 类型论确实鼓励在不需要时避免这两个原理。

首先, 每个数学家都知道, 定理使用的假设越少, 其就更加强力, 因为它能够被应用到更多的情境中。这对 AC 与 LEM 来说没什么不同: 类型论容许许多有趣的“非标准”模型, 例如层拓扑斯中像 AC 和 LEM 这样的经典原理是失效的。同伦类型论在高阶拓扑斯中也承认类似的模型, 例如[TV02, Rez05, Lur09]所研究的。因此, 如果我们避免使用这些原理, 我们证明的定理对所有这些模型而言都是内部有效的。

其次, 类型论的另一个优势就是其可计算的特征。除了作为数学基础, 类型论也是有关计算的一个形式理论, 可以被视作一个强大的编程语言。从这个角度上看, 系统的规则不能像集合论的公理那样任意选取: 它们要足够协调, 以容许所有的证明都能像程序一样被“执行”。从这一点上看, 我们并不完全了解同伦类型论所引入的一些新原理, 例如泛等性和高阶归纳类型, 但是基础大纲已经出现; 举例而言, 见 [LH12]。然而, 在很长一段时间里, 像 AC 与 LEM 这样的原理从根本上就是与可计算性相悖的, 因为它们大胆地断言特定事物的存在, 却不给出任何方法来计算它们。因此, 避免使用它们是维持类型论作为计算理论的特征的必要之举。

幸运地是, 构造性推理并没有它看起来得那么难。在某些情况下, 只需要重新表述某些定义, 一个定理就可以构造性地得到, 其证明也会更加优雅。进一步地, 在泛等基础中这种事只会更常发生。举例而言:

- (i) 在集合论基础中, 同伦论与范畴论的多处环节都需要借助选择公理来实现超限构造。而通过高阶归纳类型, 我们能够以构造性的方式直接编码这些数学结构。特别值得一提的是, 在第8章中所有的“综合”同伦论内容均不依赖 LEM 或 AC。
- (ii) 在集合论基础中, “完全忠实且本质满的函子必为范畴等价”这一命题等价于选择公理; 但在泛等公理体系中, 该命题天然成立; 参见第9章。
- (iii) 在集合论中, 为获得能典范表示集合同构类与良序集同构类的“基数”与“序数”概念, 往往需要引入复杂的迂回论述——其间可能涉及选择公理或正则公理。而依托泛等公理与高阶归纳类型, 我们只需对宇宙进行截断操作便可直接获得这类典范表示; 参见第10章。
- (iv) 在集合论基础下, 将实数定义为柯西序列等价类的方案需要依赖排中律或(可数)选择公理才能保证良定义性; 但借助高阶归纳类型, 我们可以给出一个既能规避选择公理又具备良定义性的版本; 参见第11章。

当然, 这些简化也可以作为一种证据, 最终证明新方法并非真正可构造。然而, 我们强调阅读本书时, 读者并不需要去在意或担忧其可构造性。关键在于, 对以上这些所有例子而言, 我们给出的理论版本具有独立的优势, 而不依赖于 LEM 与 AC 的真假。如果能够达成可构造性, 那将是一个额外的好处。

在当前的讨论中, 我们已经添加了诸如泛等性, 高阶归纳类型, AC 与 LEM, 人们会好奇最终系统是否还保持一致。(类型论最初的优势之一就是, 相比于集合论, 其一致性在证明论的意义下是可见的)。与任何基础系统一样, 一致性是一个相对的问题: “关于什么一致?” 简略的答案是, 归功于Voevodsky [KLV12], 这本书中所有的构造和公理都在Kan 复形范畴中有一个模型。(参见 [LS17]以了解高阶归纳类型)。因此, 已被证明的是, 它们相对于ZFC(配备上足够多的不可达基数, 其数量与嵌套泛等宇宙所需的一致)是一致的。对其一致性的更加传统的类型论解释仍在进行中。(例如参见 [LH12, BCH13])

我们将类型论操作的不同视角总结为表1:

类型论	逻辑学	集合论	同伦论
A	命题	集合	空间
$a : A$	证明	元素	点
$B(x)$	谓词	集合族	纤维丛
$b(x) : B(x)$	条件证明	元素族	截面
$0, 1$	$\perp, \top$	$\emptyset, \{\emptyset\}$	$\emptyset, *$
$A + B$	$A \vee B$	无交并	余积
$A \times B$	$A \wedge B$	有序对集	积空间
$A \rightarrow B$	$A \Rightarrow B$	函数集	函数空间
$\sum_{(x:A)} B(x)$	$\exists_{x:A} B(x)$	无交和	全空间
$\prod_{(x:A)} B(x)$	$\forall_{x:A} B(x)$	积	截面空间
Id_A	相等性 =	$\{(x, x) \mid x \in A\}$	道路空间 ¹

表 1: 类型论操作在不同视角下的比较

开放性问题

对那些有志于为这一个新的数学分支添砖加瓦的人而言, 知道该领域还有很多有意思的开放性问题是非常鼓舞人心的。

其中最急迫的是泛等公理的“可构造性”¹, 其由Voevodsky于[Voe12]中提出。类型论的基础系统服从Gentzen的自然演绎结构。逻辑连接词由它们的引入规则定义, 并且有着由其计算规则所保证的消去规则。通过使用Tait的可计算性方法——其原先被设计用于分析Gödel辩证解释——人们可以证明类型论的正则性。这反过来又蕴含了其他重要的性质。例如类型检查的可判定性(这是一个关键特性, 因为类型检查正对应着证明验证——我们有理由要求, 系统应当能够“在看到证明时识别出它”), 与所谓的“典范性质”, 其要求所有具有自然数类型的闭项能归约到一个数。在通过引入公理以拓展类型论时, 例如函数外延性公理, 或是泛等公理, 后一个性质会与引入/消去规则的统一结构一起丢失。Voevodsky已经构想出了一个精确的数学猜想, 该猜想与用泛等公理拓展的类型论的典范性相关: 给定一个自然数类型的闭项, 是否有可能找到一个数与一个该项等于该数的证明? 其中该证明本身可能用到泛等公理。更一般地, 一个重要问题在于: 是否有可能为泛等公理提供一个构造性证明? 倘若再加上其它受同伦论启发的构造, 例如高阶归纳类型, 又有什么不同? 这些问题目前仍然是开放的, 尽管各种寻找答案的方法目前都还在发展。

另一个基础的问题在于处理本质为集合(即离散空间)的类型, 例如自然数, 它们只包含平凡道路。目前, 同伦类型论只能在同伦等价意义上真正刻画空间, 这意味着这些“离散空间”就仅仅同伦等价于离散空间。从类型论的角度, 这意味着有许多道路等于自反性, 但是没有一条是判断相等于它的(关于“判断的”意义, 参见第1.1节)。尽管这种同伦不变性有着优势, 这些“无意义的”恒等项确实向论证与构造中引入了无必要的复杂性, 因此我们希望寻找一种系统性的方法将他们消去或坍塌。这将会给工作带来方便。

一个更加专业却不因此而使其重要性失色的问题是同伦类型论与高阶拓扑斯研究之间的关系。这样一种联系目前出现在高阶范畴论与同伦论的交集之中。由于两个学科间许多的熟悉之处, 愈发令人信服的是, 它们在暗中是相关联的。例如, 泛等宇宙的概念应与对象分类器相对应, 而高阶归纳类型应是局部可表性的一个“基本的”反映。更一般地, 同伦类型论应是 $(\infty, 1)$ -拓扑斯的“内部语言”, 就像直觉主义高阶逻辑是普通1-拓扑斯的内部语言。尽管有这些一般的一致性, 然而, 细节仍待被挖掘——特别地, 相容性和严格性的问题仍待被解决——而这么做毫无疑问将会给

¹译者注: 事实上, 这个问题已经由最新的立方类型论(CuTT)完成了, 参见 [CCHM16]。

两个概念都带来更为深刻的洞见。

然而迄今为止最为浩瀚的工程, 仍在于将日常数学在这一新体系中实现持续的形式化。近期在基础同伦论与范畴论中若干结论的形式化成果令人鼓舞(部分案例详见第8, 9章), 但显然, 还有许多工作亟待完成。

同伦类型论社区维护着一个网站与群体博客, 地址为 <http://homotopytypetheory.org>。同时设有一个用于学术交流的讨论邮件列表。欢迎新人加入!

如何阅读本书

这本书被分为了两个部分。第一部分, “基础”, 本部分阐述了同伦类型论的基本概念。这些数学基础既是后续具体学科发展的理论基石, 也是理解泛等基础方法论的必要前提。对于程序员而言, 这部分内容相当于“库代码”。由于泛等基础是一种全新的数学范式, 其基本概念需要一定的适应过程, 因此第一部分的论述相当详尽。

第二部分, “数学”, 由四个章节组成。其中每章都基于第一部分中的基础概念, 以展现泛等基础在四个不同数学领域的应用: 同伦论(第8章), 范畴论(第9章), 集合论(第10章)与实分析(第11)。第二部分的章节之间大体是相互独立的, 尽管有时一个章节会使用另一个章节中证明过的引理。

对那些想要真正理解泛等基础并能在其中开展研究的读者, 他们最终需要阅读并掌握第一部分的大部分内容。然而, 对于仅希望浅尝辄止、了解泛等基础及其能力的读者而言, 在接触到第二部分的“精华”之前必须先研读超过200页的内容, 难免会望而生畏。所幸的是, 要阅读第二部分的章节, 并不需要掌握第一部分的全部内容。第二部分的每一章开篇都会对该学科领域与泛等基础对其的贡献进行概述, 以及需要从第一部分预备的必要背景知识。因此, 勇敢的读者可以直接翻至其最感兴趣的主体所在章节。若读者希望比上述方式更深入地理解第二部分的一个或多个章节, 但尚未准备好通读整个第一部分, 我们在此提供第一部分的简要总结, 并附注说明哪些部分对应第二部分的哪些章节是必需的。

第1章阐述的是类型论的基本概念, 尚未涉及同伦论解读。熟悉 Martin-Löf 类型论的读者可快速浏览该章以了解我们所采用理论的具体细节。然而, 缺乏类型论经验的读者则需要仔细阅读第1章, 因为类型论与集合论等其他基础理论存在诸多细微差异。

第2章引入了类型论的同伦论视角, 以及支持此视角的基本概念, 并描述了第1章中类型论各组分同伦行为。该章还引入了泛等公理(§2.10)——这是同伦类型论两大基础创新中的第一个。因此, 该章内容非常基础, 我们鼓励所有读者阅读, 尤其是§2.1至§2.4。

第3章描述了在同伦类型论中如何表述逻辑, 及其与经典逻辑、构造性逻辑和直觉主义逻辑的联系。此处我们定义了排中律、选择公理和命题缩并公理(尽管在本书后续大部分内容中我们无需假设这些公理), 同时介绍了对于表述传统逻辑至关重要的命题截断。此章是阅读第10章和第11章的必要背景, 对第9章重要性较低, 对第8章则非必需。

第4章和第5章详细研究了两个专题: 等价性(及相关概念)和广义归纳定义。尽管这些主题本身很重要且能加深对同伦类型论的理解, 但在第二部分大部分内容中并非必需。仅在第4章中的少数引理被零星使用, 而§5.1、§5.6和§5.7中的一般性讨论有助于为理解第6章提供必要的直观背景。第5.7节讨论的广义归纳定义也在第10章和第11章中的少数地方有所使用。

第6章通过诸多示例引入了同伦类型论的第二个基础创新——高阶归纳类型——。高阶归纳类型是第8章的主要研究对象, 并且某些特定的高阶归纳类型在第10章和第11章中扮演重要角色。它们对第9章则非必需, 尽管在§9.9中使用了一个例子。

最后, 第7章讨论了同伦 n -类型及相关概念, 如 n -连通类型。这些概念对第8章很重要, 但在第二部分其余章节中不那么重要, 尽管其中一些引理在 $n = -1$ 时的特例在§10.1中被用到。

第一部分到此结束。如前所述, 第二部分由四个很大程度上无关的章节组成, 每章都描述了泛等基础在某一个特定学科的应用。

在第二部分的各章中, 第8章(同伦论)或许是最具革命性的。泛等基础采用了一种非常不同的“综合”的方式来处理同伦论, 其中同伦类型是基本对象(即类型本身), 而非通过拓扑空间或其他

集合论模型构造而来。这使得代数拓扑中的经典定理有了新的证明方式,我们从中选取了一些示例,从 $\pi_1(S^1) = \mathbb{Z}$ 到Freudenthal纬悬定理。

在第9章(范畴论)中,我们发展了一些基本的(1-)范畴理论,遵循泛等公理的原则,即相等即同构。这产生了一个良好的性质:确保所有定义和构造在范畴等价下自动满足不变性。事实上,等价的范畴是相等的,正如等价的类型是相等的一样。(这也与高阶范畴论和高阶拓扑斯理论存在联系。)

第10章(集合论)研究泛等基础中的集合。集合范畴具有其通常的性质,从而为任何不需要同伦或高阶范畴结构的数学提供了基础。我们还观察到,泛等公理使得基数和序数的处理更为优雅,并且高阶归纳类型产生了一个满足Zermelo–Fraenkel集合论一般公理的累积层级。

在第11章(实数)中,我们总结了Dedekind实数的构造,然后指出高阶归纳类型允许一种Cauchy实数的定义,该定义避免了构造性数学中一些相关的问题。接着我们概述了对Conway超实数的类似处理方法。

本书的每一章都以“注释”节作结,该节尽可能收集了历史评论、文献引用和成果归属说明。我们还在每章末尾附有习题,以帮助读者熟悉在泛等基础中进行数学研究的方式。

最后,请记住本书是由大量参与者通过大规模协作努力撰写而成的。我们已尽力确保术语和符号的一致性,并将数学内容以逻辑流畅的线性结构呈现,但很可能仍存在一些不完美之处。我们恳请读者对此类疏漏予以谅解,并欢迎为下一版的改进提出建议。

PART I

基础

第 1 章

类型论

1.1 类型论 vs 集合论

同伦类型论(与其他的基础一样)是数学的一种基础语言,即 Zermelo–Fraenkel 集合论的一种替代方案。然而,它在几个重要方面的行为方式与集合论不同,这可能需要一些时间来适应。为了仔细解释这些差异,与本书其余部分相比,我们需要在此处更加形式化。如引言中所述,我们的目标是非形式化地书写类型论;但对于习惯于集合论的数学家来说,在开始时更加精确有助于避免一些常见的误解和错误。

我们注意到,集合论基础有两个“层次”:一阶逻辑的演绎系统,以及在该系统内部形式化的特定理论(例如 ZFC)的公理。因此,集合论不仅仅是关于集合的,而是关于集合(第二层次的对象)与命题(第一层次的对象)之间的相互作用。

相反,类型论是它自己的演绎系统:它不需要在任何上层结构(如一阶逻辑)的内部形式化。与集合论的两个基本概念(集合和命题)不同,类型论有一个基本概念:类型。命题(我们可以证明、证伪、假设、否定等的陈述¹)根据第 8 页的表 1 中所示的对应关系,被等同于特定的类型。因此,证明定理的数学活动被等同于构造对象的数学活动的一个特例——在这个意义下,就是构造一个表示命题的类型的居留元。

这使我们想到类型论和集合论之间的另一个区别,但为了解释它,我们必须稍微谈谈一般的演绎系统。非正式地说,演绎系统是一个**规则**的集合,用于推导称为**判断**的东西。如果我们将演绎系统视为一个形式游戏,那么判断就是我们通过遵循游戏规则而达到的游戏中的“位置”。我们也可以将演绎系统视为一种代数理论,在这种情况下,判断是元素(如群中的元素),而演绎规则是运算(如群的乘法)。从逻辑的观点来看,判断可以被视为“外部”陈述,存在于元理论中,与理论本身的“内部”陈述相对。

在一阶逻辑推理系统中(集合论所基于的推理系统),只存在一类判断:给定的一个命题存在证明。这就是说,每个命题 A 都会产生一个判断“ A 存在证明”,而所有的判断都具有该形式。一阶逻辑中形如“从 A 和 B 推断出 $A \wedge B$ ”的规则,实际上是一个“证明构造”的规则,其断言,只要给定判断“ A 存在证明”与“ B 存在证明”,我们可以就推理出“ $A \wedge B$ 存在证明”。需要注意的是,判断“ A 存在证明”与 A 本身处在不同的层级,后者是理论的内部陈述。

与“ A 存在证明”相似,类型论的基础判断写作“ $a : A$ ”,读作“项 a 具有类型 A ”,或者更宽泛地读作“ a 是 A 的元素”(或在同伦类型论中读作“ a 是 A 中一点”)。若 A 是一个表示命题的类型,那么 a 被称作对 A 可证明性的一个见证,或是 A 真值的一个证据(或甚至为 A 的一个证明,但我们会避免使用这个令人困惑的术语)。在这种情况下,当一阶逻辑中相似的判断“ A 存在证明”是可以被导出的,那

¹令人困惑的是,将“命题”一词与“定理”同义使用也是一种常见做法(可追溯到欧几里得)。我们将自己局限于逻辑学家的用法,根据这种用法,命题是一个易于证明的陈述,而定理(或“引理”或“推论”)是一个已经被证明的陈述。因此“ $0 = 1$ ”和它的否定“ $\neg(0 = 1)$ ”都是命题,但只有后者是定理。

么判断 $a : A$ 在类型论中(对某些 a)是可以被精确地导出的(模去我们假设的公理和数学编码的差异,正如我们将在全书讨论的那样)。

另一方面,如果类型 A 被更多地视作一个集合而非一个命题(尽管正如我们将看到的,这种区分可能变得模糊),那么 $a : A$ 可被视为类似于集合论陈述 $a \in A$ 。然而,存在一个本质区别: $a : A$ 是一个判断,而 $a \in A$ 是一个命题。特别地,在类型论内部工作时,我们不能做出诸如“若 $a : A$,则 $b : B$ 不成立”这样的陈述,也不能“反驳”判断 $a : A$ 。

理解这一点的一个好方法是:在集合论中,“属于”是一种关系,它可能在两个预先存在的对象 a 和 A 之间成立,也可能不成立;而在类型论中,我们不能孤立地谈论一个元素 a :每个元素根据其本质都是某个类型的元素,并且该类型(一般而言)是唯一确定的。因此,当我们非正式地说“设 x 为一个自然数”时,在集合论中,这是“设 x 为一个事物并假设 $x \in \mathbb{N}$ ”的简写,而在类型论中“设 $x : \mathbb{N}$ ”是一个原子陈述:我们不能在未指定其类型的情况下引入一个变量。

乍看之下,这似乎是一个不便的限制,但可以说,它更接近“设 x 为一个自然数”的直观数学含义。在实践中,似乎每当我们实际上需要 $a \in A$ 是一个命题而非判断时,总存在一个背景集合 B ,已知 a 是其元素且 A 是其子集。这种情况在类型论中也容易表示,只需将 a 视为类型 B 的一个元素,而将 A 视为 B 上的一个谓词;参见§3.5。

类型论与集合论之间的最后一个区别是对相等的处理。数学中熟悉的概念是一个命题:例如,我们可以反驳一个等式或假设一个等式作为前提。由于在类型论中,命题是类型,这意味着相等是一个类型:对于元素 $a, b : A$ (即同时有 $a : A$ 和 $b : A$),我们有一个类型 $a =_A b$ 。(当然,在同伦类型论中,这个相等命题可能以不熟悉的方式行为:参见§1.12和第2章,以及本书其余部分)。当 $a =_A b$ 有居留元时,我们说 a 和 b 是(命题)相等的。

然而,在类型论中还需要一种相等判断,它存在于与判断 $x : A$ 相同的层次。这被称为**判断相等**或**定义相等**,我们将其写为 $a \equiv b : A$ 或简写为 $a \equiv b$ 。将其理解为“按定义相等”是有帮助的。例如,如果我们通过方程 $f(x) = x^2$ 定义一个函数 $f : \mathbb{N} \rightarrow \mathbb{N}$,那么表达式 $f(3)$ 就按定义等于 3^2 。在理论内部,否定或假设一个按定义的相等是没有意义的;我们不能说“如果 x 按定义等于 y ,那么 z 就不按定义等于 w ”。两个表达式是否按定义相等只是一个展开定义的问题;特别地,它是算法上可判定的(尽管该算法必然是元理论的,而非理论内部的)。

随着类型论变得更为复杂,判断相等可能比这更为微妙,但这是一个很好的最初直觉。或者,如果我们将演绎系统视为一个代数理论,那么判断相等就是该理论中的简单相等,类似于群中元素之间的相等——唯一潜在的混淆是,在类型论的演绎系统内部也有一个对象(即类型 $a = b$),在其内部行为上,它被视作一种“相等”概念。

我们希望有一个判断的相等性概念的原因在于其可以控制其他形式的判断,即形如 $a : A$ 的判断。举例而言,假设我们已经有了对 $3^2 = 9$ 的证明,即,我们得到了某个 p ,对其有判断 $p : (3^2 = 9)$ 。那么同一个 p 就应该被算作是一个关于 $f(3) = 9$ 的证明,其中 $f(3)$ 依定义等于 3^2 。表示这一点的最好方法是通过一个规则,其声明如果给定判断 $a : A$ 与 $A \equiv B$,我们就可以导出判断 $a : B$ 。

因此,对我们而言,类型论将是一个基于以下两种判断的推理系统。

判断	含义
$a : A$	“ a 是类型 A 的对象”
$a \equiv b : A$	“ a 与 b 是类型 A 中依定义相等的对象”

在引入定义相等——即将一个事物定义为等于另一个事物——时,我们将使用符号 $“\equiv”$ 。因此,上述函数 f 的定义将写为 $f(x) := x^2$ 。

由于判断不能被组合成更复杂的陈述,符号 $“:”$ 和 $“\equiv”$ 的结合性比任何其他符号都更弱。²因此,例如,“ $p : x = y$ ”应被解析为“ $p : (x = y)$ ”,这是合理的,因为“ $x = y$ ”是一个类型,而不

²在形式化的类型论中,逗号和推出符号的结合性可能更弱。例如, $x : A, y : B \vdash c : C$ 被解析为 $((x : A), (y : B)) \vdash (c : C)$ 。然而,在本书中,我们直到第A章才会使用这种表示法。

应解析为“ $(p : x) = y$ ”，后者是无意义的，因为“ $p : x$ ”是一个判断，不能等于任何东西。类似地，“ $A \equiv x = y$ ”只能被解析为“ $A \equiv (x = y)$ ”，尽管在这种极端情况下，无论如何都应添加括号以帮助阅读理解。此外，后面我们将沿用串联等式的常见表示法——例如，写 $a = b = c = d$ 表示“ $a = b$ 且 $b = c$ 且 $c = d$ ，因此 $a = d$ ”——并且我们将在这种链条中包含判断相等。通常，上下文足以使我们的意图清晰。

这里或许也是提及常见数学符号“ $f : A \rightarrow B$ ”(表示 f 是从 A 到 B 的函数)的适当时机，它可以被视为一个类型判断，因为我们使用“ $A \rightarrow B$ ”作为从 A 到 B 的函数类型的记号(这是类型论中的标准做法；参见§1.4)。

判断可能依赖于形式为 $x : A$ 的假设，其中 x 是变量而 A 是一个类型。例如，我们可以在假设 $m, n : \mathbb{N}$ 下构造一个对象 $m + n : \mathbb{N}$ 。另一个例子是：假设 A 是一个类型， $x, y : A$ ，且 $p : x =_A y$ ，我们可以构造一个元素 $p^{-1} : y =_A x$ 。所有此类假设的集合称为上下文；从拓扑的观点来看，它可以被视为一个“参数空间”。实际上，从技术上讲，上下文必须是一个有序的假设列表，因为后面的假设可能依赖于前面的假设：只有在类型 A 中出现的任何变量作出假设之后，我们才能做出假设 $x : A$ 。

如果假设 $x : A$ 中的类型 A 表示一个命题，则该假设是如下假说的类型论版本：我们假设命题 A 成立。当类型被视为命题时，我们可以省略其证明的名称。因此，在上面的第二个例子中，我们可以改为说：假设 $x =_A y$ ，我们可以证明 $y =_A x$ 。然而，由于我们正在做“证明相关”的数学，我们将频繁地将证明作为对象来引用。例如，在上面的例子中，我们可能想要确定， p^{-1} ，与传递性和自反性的证明一起，其行为类似于群胚；参见第2章。

注意，在假设一词的这种含义下，我们可以假设一个命题相等(通过假设一个变量 $p : x = y$)，但不能假设一个判断相等 $x \equiv y$ ，因为它不是一个可以拥有元素的类型。然而，我们可以做另一件看起来有点像假设判断相等的事情：如果我们有一个涉及变量 $x : A$ 的类型或元素，那么我们可以用任何特定元素 $a : A$ 代入 x ，以获得更具体的类型或元素。我们有时会使用诸如“现在假设 $x \equiv a$ ”的语言来指代这种代入过程，即使它在技术意义上不是上面介绍的假设。

同样地，我们也不能证明一个判断相等，因为它不是一个我们可以展示见证者的类型。然而，我们有时会在定理中陈述判断相等，例如“存在 $f : A \rightarrow B$ 使得 $f(x) \equiv y$ ”。这应被视为做出两个独立的判断：首先我们对某个元素 f 做出判断 $f : A \rightarrow B$ ，然后我们做出额外的判断 $f(x) \equiv y$ 。

在本章的剩余部分，我们尝试给出类型论的非形式化阐述，其足以满足本书的目的；我们在第A章给出更形式化的说明。除了一些相当明显的规则(例如判断相等的事物总是可以相互替换)，类型论的规则可以分组为类型构造子。每个类型构造子包括一种构造类型的方法(可能利用先前构造的类型)，以及该类型元素的构造和行为规则。在大多数情况下，这些规则遵循相当可预测的模式，但我们不试图在此使其精确；然而可参见§1.5的开头以及第5章。

本章所介绍的类型论的一个重要方面是它完全由规则组成，而不包含任何公理。在用判断来描述演绎系统时，通过规则，我们能够从一组判断中推断出一个判断，而公理则被初始给定。如果我们将演绎系统视为一个形式游戏，那么规则就是游戏规则，而公理则是起点。如果我们将演绎系统视为一个代数理论，那么规则就是理论的操作，而公理则是该理论某个特定自由模型的生成元。

在集合论中，唯一的规则是一阶逻辑的规则(例如允许我们从“ A 有一个证明”和“ B 有一个证明”推导出“ $A \wedge B$ 有一个证明”的规则)：所有关于集合行为的信息都包含在公理中。与之相对的，在类型论中，通常是规则包含了所有信息，而不需要任何公理。例如，在§1.5中我们将看到，存在一条规则，其允许我们从“ $a : A$ ”和“ $b : B$ ”推导出判断“ $(a, b) : A \times B$ ”，而在集合论中，类似的陈述将是配对公理(的一个推论)。

仅使用规则来形式化类型论，其优势在于规则的“程序性”。事实上，“程序性”使得类型论有可能具备一些良好的计算性质(尽管不能自动确保)，例如“典范性”。然而，虽然这种风格适用于传统的类型论，但我们尚未理解如何以这种方式形式化同伦类型论所需的一切。特别地，在§2.9、§2.10和第6章中，我们将不得不通过引入额外的公理来扩充本章介绍的类型论规则，特别是泛等公理。然而，在本章中，我们将讨论范围局限于传统的、基于规则的类型论。

1.2 函数类型

给定类型 A 与 B , 我们可以构造定义域为 A , 陪域为 B 的函数类型 $A \rightarrow B$ 。我们有时也会将函数称为映射。与集合论不同, 函数并不被定义为函数关系, 而是类型论的基础概念。我们通过规定如何使用函数, 如何构造它们, 以及它们能引出怎样的相等性来阐释函数类型。

给定一个函数 $f : A \rightarrow B$ 与定义域中的一个元素 $a : A$, 我们可以应用该函数来得到陪域中的一个元素 B , 将其记为 $f(a)$ 并称之为函数 f 在 a 处的函数值。类型论中通常会省略括号, 并将 $f(a)$ 简单记为 $f a$, 我们有时也会遵循这一惯例。

但是我们该如何构造 $A \rightarrow B$ 的元素呢? 有两种等价的方式: 一种是通过直接定义, 另一种是使用 λ 抽象。通过直接定义引入函数, 指的是我们通过给函数命名(比方说 f)来引入它, 并通过给出一个等式

$$f(x) \equiv \Phi \quad (1.2.1)$$

来定义 $f : A \rightarrow B$, 其中 x 是一个变量而 Φ 是一个可以使用 x 的表达式。为了使其有效, 我们必须检查在假设 $x : A$ 的情况下有 $\Phi : B$ 。

现在我们可以通过将 Φ 中的变量 x 替换为 a 来计算 $f(a)$ 。例如, 考虑函数 $f : \mathbb{N} \rightarrow \mathbb{N}$, 它由 $f(x) \equiv x + x$ 定义。(我们将在§1.9中定义 $\mathbb{N}$ 和 $+$ 。)那么 $f(2)$ 判断相等于 $2 + 2$ 。

如果我们不想为函数命名, 我们可以使用 λ 抽象。给定一个如上所述可能使用 $x : A$ 的、类型为 B 的表达式 Φ , 我们写作 $\lambda(x : A). \Phi$ 来表示由 (1.2.1) 定义的同一个函数。因此, 我们有

$$(\lambda(x : A). \Phi) : A \rightarrow B.$$

对于前一段中的例子, 我们有类型判断

$$(\lambda(x : \mathbb{N}). x + x) : \mathbb{N} \rightarrow \mathbb{N}.$$

作为另一个例子, 对于任何类型 A 和 B 以及任何元素 $y : B$, 我们有一个常函数 $(\lambda(x : A). y) : A \rightarrow B$ 。

我们通常省略 λ 抽象中变量 x 的类型, 写作 $\lambda x. \Phi$, 因为从函数 $\lambda x. \Phi$ 具有类型 $A \rightarrow B$ 的判断中可以推断出 $x : A$ 。按照约定, 绑定变量 “ $\lambda x.$ ” 的“作用域” 是整个表达式的其余部分, 除非用括号将其括起。因此, 例如, $\lambda x. x + x$ 应该解析为 $\lambda x. (x + x)$, 而不是 $(\lambda x. x) + x$ (在这种情况下, 后者本身就是类型错误的)。

另一个等价的记号是

$$(x \mapsto \Phi) : A \rightarrow B.$$

我们有时也会在表达式 Φ 中使用一个空的 “ $-$ ” 来代替变量, 以表示一个隐式的 λ -抽象。例如, $g(x, -)$ 是书写 $\lambda y. g(x, y)$ 的另一种方式。

现在, λ -抽象本身即是函数, 因此我们可以将它应用于参数 $a : A$ 。于是我们有以下计算规则³, 这是一个定义等式:

$$(\lambda x. \Phi)(a) \equiv \Phi'$$

其中 Φ' 是表达式 Φ 中所有 x 的出现都被 a 替换后的结果。继续上面的例子, 我们有

$$(\lambda x. x + x)(2) \equiv 2 + 2.$$

注意, 从任意函数 $f : A \rightarrow B$, 我们可以构造一个 λ 抽象函数 $\lambda x. f(x)$ 。依定义, 这是“将 f 应用于其参数的函数”, 我们认为它与 f 定义相等:⁴

$$f \equiv (\lambda x. f(x)).$$

³该等式通常被称为 β -转换 或 β -规约。

⁴该等式通常被称为 η -转换 或 η -展开。

这个等式被称为**函数类型的唯一性原理**, 因为它表明 f 由其值唯一决定。

通过使用 λ -抽象, 我们可以简化带有显式参数的, 由直接定义引入函数的方法: 即, 我们可以将

$$f(x) := \Phi$$

的定义读作

$$f := \lambda x. \Phi.$$

当进行涉及变量的计算时, 我们必须小心地用同样涉及变量的表达式替换变量, 因为我们希望保持表达式的绑定结构。这里所说的绑定结构是指一种在变量的引入位置与使用位置间的不可见链接。这种链接由诸如 λ 、 Π 和 Σ (我们很快就会遇到后者)的绑定符生成。例如, 考虑定义 $f : \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$

$$f(x) := \lambda y. x + y.$$

如果我们在某处假设了 $y : \mathbb{N}$, 那么 $f(y)$ 是什么? 在 $f(x)$ 的定义式“ $\lambda y. x + y$ ”中天真地用 y 替换所有位置上的 x , 从而得到 $\lambda y. y + y$ 是错误的做法, 因为这意味着 y 被**捕获**了。之前, 被替换的 y 指的是我们的假设, 但现在它指的是 λ -抽象的参数。因此, 这种天真的替换会破坏绑定结构, 允许我们执行语义上不可靠的计算。

但在这个例子中, $f(y)$ 究竟是什么? 注意, 绑定(或“哑”)变量 (如表达式 $\lambda y. x + y$ 中的 y)只有局部意义, 并且可以一致地被任何其他变量替换, 同时保持绑定结构。实际上, $\lambda y. x + y$ 被声明为判断相等⁵于 $\lambda z. x + z$ 。由此可知, $f(y)$ 判断相等于 $\lambda z. y + z$, 这就回答了这一问题。(除了 z , 任何与 y 不同的变量都可以使用, 并得到相等的结果。)

当然, 这任何数学家都应该对此感到熟悉: 这与以下现象相同: 如果 $f(x) := \int_1^2 \frac{dt}{x-t}$, 那么 $f(t)$ 不是 $\int_1^2 \frac{dt}{t-t}$ 而是 $\int_1^2 \frac{ds}{t-s}$ 。 λ -抽象绑定哑变量的方式与积分如出一辙。

我们已经了解如何定义单变量函数。一种定义多变量函数的方式是使用Cartesian积, 这将在后面介绍; 我们把一个参数为 A 和 B , 结果为 C 的函数赋予类型 $f : A \times B \rightarrow C$ 。然而, 还有一种被称为**柯里化**的方法可以避免使用乘积类型 (以数学家Haskell Curry命名)。

柯里化的思想是将一个有两个输入 $a : A$ 和 $b : B$ 的函数表示为一个接受单个输入 $a : A$ 并返回另一个函数的函数, 而这个函数接受第二个输入 $b : B$ 并返回结果。也就是说, 我们认为双变量函数属于一个迭代函数类型, $f : A \rightarrow (B \rightarrow C)$ 。我们也可以省略括号并写作, 如 $f : A \rightarrow B \rightarrow C$, 其默认向右结合。于是, 给定 $a : A$ 和 $b : B$, 我们可以将 f 应用于 a , 然后将结果应用于 b , 得到 $f(a)(b) : C$ 。为了避免括号的泛滥, 我们允许将 $f(a)(b)$ 写作 $f(a, b)$, 即使没有涉及到乘积。当完全省略函数参数周围的括号时, 我们把 $(f a) b$ 写作 $f a b$, 其默认向左结合, 从而让 f 以正确的顺序应用于其参数。

于是, 带有显式参数的定义也适用于该情况: 我们可以通过给出一个等式

$$f(x, y) := \Phi$$

来定义一个命名函数 $f : A \rightarrow B \rightarrow C$, 其中 $\Phi : C$ 假设 $x : A$ 和 $y : B$ 。使用 λ -抽象的话, 这对应于

$$f := \lambda x. \lambda y. \Phi,$$

这也可以写成

$$f := x \mapsto y \mapsto \Phi.$$

我们也可以通过多个空白来隐式抽象多个变量, 例如 $g(-, -)$ 表示 $\lambda x. \lambda y. g(x, y)$ 。通过直接拓展刚刚所述内容, 我们可以对三个或更多参数的函数进行柯里化。

⁵该等式通常被称为 α -转换。

1.3 宇宙与族

目前为止, 我们都在不形式化地使用“ A 为类型”的表达. 通过引入**宇宙**, 我们可以让其更加精确. 宇宙是其元素为类型的类型. 与朴素集合论类似, 我们会希望有一个包含所有类型的宇宙 $\mathcal{U}_\infty$, 其也包含它自身(也就是有 $\mathcal{U}_\infty : \mathcal{U}_\infty$). 然而, 与集合论一样, 这是不可靠的, 即我们能从中推断出每个类型都是被居留的, 包括那些表示命题假的空类型(见§1.7). 例如, 使用集合的树形表示, 我们可以直接编码Russell悖论 [Coq92a].

为了避免悖论, 我们引入宇宙层级

$$\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$$

其中每个宇宙 $\mathcal{U}_i$ 都是下一个宇宙 $\mathcal{U}_{i+1}$ 的一个元素. 此外, 我们假设我们的宇宙是**累积的**, 即第 i 个宇宙的所有元素也是第 $(i+1)$ 个宇宙的元素, 也就是说如果 $A : \mathcal{U}_i$, 那么也有 $A : \mathcal{U}_{i+1}$. 这很方便, 但有一个稍微烦人的后果, 即元素不再有唯一的类型, 并且在其他一些方面也有些棘手, 这里我们无需关注: 参见注释.

当 A 居于某个宇宙 $\mathcal{U}_i$ 中时, 我们称其为一个类型. 我们通常希望避免显式提及层级 i , 而只是假设层级可以一致地分配; 因此我们可以省略层级, 将其写作 $A : \mathcal{U}$. 这样我们甚至可以写出 $\mathcal{U} : \mathcal{U}$, 它可以被读作 $\mathcal{U}_i : \mathcal{U}_{i+1}$, 其中索引被隐去. 以这种风格书写宇宙被称为**典型歧义**. 这种风格方便但危险, 因为它允许我们写出看起来有效的证明, 而这些证明会重现自指的悖论. 如果怀疑论证是否正确, 检查的方法就是尝试为其中出现的所有宇宙一致地分配层级. 在提前设定某个宇宙 $\mathcal{U}$ 时, 我们可以将属于 $\mathcal{U}$ 的类型称为**小类型**.

为了模拟在给定类型 A 上变化的类型集合, 我们使用陪域为宇宙的函数 $B : A \rightarrow \mathcal{U}$. 这些函数被称为**类型族**(有时也称为**依值类型**); 它们对应于集合论中使用的集合族.

类型族的一个例子是有限集族 $\text{Fin} : \mathbb{N} \rightarrow \mathcal{U}$, 其中 $\text{Fin}(n)$ 是一个恰好有 n 个元素的类型. (我们还不能定义族 Fin ——事实上, 我们甚至还没有引入它的定义域 $\mathbb{N}$ ——但我们很快就能做到; 参见§1.9.) 我们可以用 $0_n, 1_n, \dots, (n-1)_n$ 表示 $\text{Fin}(n)$ 的元素, 其中的下标用于强调当 n 与 m 不同时, $\text{Fin}(n)$ 与 $\text{Fin}(m)$ 的元素是不同的, 并且它们都不同于普通的自然数(我们将在§1.9中引入).

一个更平凡(但非常重要)的类型族例子是某个类型 $B : \mathcal{U}$ 上的**常类型族**. 它当然是常函数 $(\lambda(x:A). B) : A \rightarrow \mathcal{U}$.

作为一个反例, 在我们这个版本的类型论中, 没有类型族“ $\lambda(i:\mathbb{N}). \mathcal{U}_i$ ”. 确实, 没有足够大的宇宙可以作为其陪域. 此外, 我们甚至不认为宇宙 $\mathcal{U}_i$ 的索引 i 是类型论中的自然数 $\mathbb{N}$ (后者将在§1.9中引入).

1.4 依值函数类型 (Π -类型)

在类型论中, 我们经常使用一个广义版本的函数类型, 称为 **Π -类型**或**依值函数类型**. 一个 Π -类型的元素是这样一种函数, 随着函数应用的定义域中的元素的变化, 其陪域的类型可以随之变化, 因此称为**依值函数**. “ Π -类型”这一名称源于该类型也可以被视为给定类型上的Cartesian积.

给定一个类型 $A : \mathcal{U}$ 和一个类型族 $B : A \rightarrow \mathcal{U}$, 我们可以构造依值函数的类型 $\prod_{(x:A)} B(x) : \mathcal{U}$. 这个类型有许多替代记号, 例如

$$\prod_{(x:A)} B(x) \quad \prod_{(x:A)} B(x) \quad \Pi(x:A), B(x).$$

如果 B 是一个常类型族, 那么依值函数类型就是普通的函数类型:

$$\prod_{(x:A)} B \equiv (A \rightarrow B).$$

实际上, Π -类型的所有构造都是普通函数类型上相应构造的推广。

我们可以通过显式定义来引入依值函数: 若要定义 $f : \prod_{(x:A)} B(x)$, 其中 f 是要定义的依值函数的名称, 我们需要一个可能涉及变量 $x : A$ 的表达式 $\Phi : B(x)$, 并写作

$$f(x) := \Phi \quad \text{对于 } x : A.$$

另一种方法是使用 λ -抽象

$$\lambda x. \Phi : \prod_{x:A} B(x). \quad (1.4.1)$$

与非依值函数一样, 我们可以将依值函数 $f : \prod_{(x:A)} B(x)$ 应用于一个参数 $a : A$, 得到一个元素 $f(a) : B(a)$ 。等式与普通函数类型相同, 即我们有如下计算规则: 给定 $a : A$, 我们有 $f(a) \equiv \Phi'$ 和 $(\lambda x. \Phi)(a) \equiv \Phi'$, 其中 Φ' 是通过将 Φ 中所有 x 替换为 a 得到的(一如既往地, 要避免变量捕获)。类似地, 对于任意 $f : \prod_{(x:A)} B(x)$, 我们有唯一性原理 $f \equiv (\lambda x. f(x))$ 。

例如, 回忆§1.3中的类型族 $\text{Fin} : \mathbb{N} \rightarrow \mathcal{U}$, 其值是标准有限集, 元素为 $0_n, 1_n, \dots, (n-1)_n : \text{Fin}(n)$ 。那么有一个依值函数 $\text{fmax} : \prod_{(n:\mathbb{N})} \text{Fin}(n+1)$, 它返回每个非空有限类型的“最大”元素, $\text{fmax}(n) := n_{n+1}$ 。就像 Fin 本身的情况一样, 我们现在还不能定义 fmax , 但很快就能做到; 参见练习1.9。

我们现在可以定义的另一类重要的依值函数类型是那些在给定宇宙上具有多态的函数。多态函数是这样一种函数: 其将类型作为其参数之一, 然后作用于该类型(或由它构造的其他类型)的元素。

一个例子是多态恒等函数 $\text{id} : \prod_{(A:\mathcal{U})} A \rightarrow A$, 我们通过 $\text{id} := \lambda(A:\mathcal{U}). \lambda(x:A). x$ 定义它。

(类似于 λ 抽象, Π 会自动作用于表达式的其余部分, 除非被分隔; 因此 $\text{id} : \prod_{(A:\mathcal{U})} A \rightarrow A$ 表示 $\text{id} : \prod_{(A:\mathcal{U})} (A \rightarrow A)$ 。这种约定虽然在数学中不常见, 但在类型论中很常见。)

我们有时将依值函数的某些参数写作下标。例如, 我们可以等价地通过 $\text{id}_A(x) := x$ 来定义多态恒等函数。

此外, 如果一个参数可以从上下文推断, 我们可以完全省略它。例如, 如果 $a : A$, 那么写作 $\text{id}(a)$ 是明确的, 因为 id 必须表示 id_A 以便它适用于 a 。

我们有另一个不那么平凡的多态函数例子。对(柯里化的)双参数函数, 考虑对其参数顺序的“交换”操作:

$$\text{swap} : \prod_{(A:\mathcal{U})} \prod_{(B:\mathcal{U})} \prod_{(C:\mathcal{U})} (A \rightarrow B \rightarrow C) \rightarrow (B \rightarrow A \rightarrow C).$$

我们可以通过以下方式定义它

$$\text{swap}(A, B, C, g) := \lambda b. \lambda a. g(a)(b).$$

我们也可以等价地将类型参数写作下标:

$$\text{swap}_{A,B,C}(g)(b, a) := g(a, b).$$

注意, 正如我们对普通函数所做的那样, 我们使用柯里化来定义具有多个参数的依值函数(例如 swap)。然而, 在依值的情况下, 第二个定义域可能依赖于第一个, 并且值域可能依赖于两者。也就是说, 给定 $A : \mathcal{U}$ 和类型族 $B : A \rightarrow \mathcal{U}$ 以及 $C : \prod_{(x:A)} B(x) \rightarrow \mathcal{U}$,

当 B 是常数且等于 A 时, 我们可以简化记号并写作 $\prod_{(x,y:A)}$; 例如, swap 的类型也可以写作

$$\text{swap} : \prod_{A,B,C:\mathcal{U}} (A \rightarrow B \rightarrow C) \rightarrow (B \rightarrow A \rightarrow C).$$

最后, 给定 $f : \prod_{(x:A)} \prod_{(y:B(x))} C(x, y)$ 和参数 $a : A$ 与 $b : B(a)$, 我们有 $f(a)(b) : C(a, b)$, 和之前一样, 我们写作 $f(a, b) : C(a, b)$ 。

1.5 乘积类型

给定类型 $A, B : \mathcal{U}$, 我们引入类型 $A \times B : \mathcal{U}$, 称为它们的**Cartesian积**. 我们同时引入一个作为乘积单位元的类型, 称为**单位类型** $\mathbf{1} : \mathcal{U}$. 我们认为 $A \times B$ 的元素为有序对 $(a, b) : A \times B$, 其中 $a : A$ 且 $b : B$, 而 $\mathbf{1}$ 的唯一元素为某个特定对象 $\star : \mathbf{1}$. 然而, 这与集合论不同. 在集合论中, 我们将有序对定义为特定的集合, 并将它们收集到Cartesian积中, 在类型论中, 有序对是一个像函数一样的基础概念。

Remark 1.5.1. 在类型论中引入新类型有一个通用模式. 我们已经在§1.2和§1.4中看到了这种模式⁶, 因此其一般形式值得强调. 要指定一个类型, 我们需要指定:

(i) 如何通过**形成规则**形成这种新类型.

(例如, 当 A 与 B 是类型时, 我们可以形成函数类型 $A \rightarrow B$. 当 A 是类型且对 $x : A$ 有类型 $B(x)$ 时, 我们可以形成依值函数类型 $\prod_{(x:A)} B(x)$.)

(ii) 如何构造该类型的元素. 这些称为类型的**构造子**或**引入规则**

(例如, 函数类型有一个构造子, 即 λ -抽象. 回忆像 $f(x) := 2x$ 这样的直接定义, 可以等价地表述为 λ -抽象 $f := \lambda x. 2x$.)

(iii) 如何使用该类型的元素. 这些称为类型的**消去子**或**消去规则**.

(例如, 函数类型有一个消去子, 即函数应用.)

(iv) 一条**计算规则**⁷, 它表达消去子如何作用于构造子.

(例如, 对于函数, 计算规则断言 $(\lambda x. \Phi)(a)$ 判断相等于将 a 代入 Φ 中的 x 得到的结果.)

(v) 一个可选的**唯一性原理**⁸, 它表达映入/映出该类型的映射的唯一性. 对于某些类型, 唯一性原理通过断言该类型的每个元素由对其应用消去子的结果唯一确定, 并且可以通过应用构造子, 从这些结果重构出映入该类型的映射——从而表达了构造子如何作用于消去子, 与计算规则对偶. (例如, 对于函数, 唯一性原理说任何函数 f 判断相等于“展开后的”函数 $\lambda x. f(x)$, 因此由其值唯一确定.) 对于其他类型, 唯一性原理说每个从该类型出发的映射(函数)由某些数据唯一确定. (一个例子是在§1.7中引入的余积类型, 其唯一性原理在§2.15中提到.) 当唯一性原理不作为判断相等的规则时, 通常, 仍然可以从该类型的其他规则出发, 将其证明为一个命题等式. 在这种情况下, 我们称它为**命题性唯一性原理**. (在后续章节中, 我们也会偶尔遇到命题性计算规则.)

§A.2中的推理规则相应地组织和命名; 例如, 参见§A.2.4, 其中每种可能性都得到了实现.

构造有序对的方式是显然的: 给定 $a : A$ 和 $b : B$, 我们可以构造 $(a, b) : A \times B$. 类似地, 有一种唯一的方式构造 $\mathbf{1}$ 的元素, 即我们有 $\star : \mathbf{1}$. 我们的期望是“ $A \times B$ 的每个元素都是一个有序对”, 这是乘积类型的唯一性原理; 我们并不将其作为类型论的一条规则, 但将在稍后证明其是一个命题等式。

现在, 我们该如何使用有序对, 即如何定义从乘积类型出发的函数? 让我们首先考虑非依值函数 $f : A \times B \rightarrow C$ 的定义. 由于我们希望使 $A \times B$ 的唯一元素为有序对, 因此我们希望能够通过规定 f 应用于有序对 (a, b) 时的结果来对这样的函数进行定义. 我们可以通过提供一个函数 $g : A \rightarrow B \rightarrow C$ 来规定这些结果. 因此, 我们将引入一条新规则(乘积的消去规则), 它声明对于任何这样的 g , 我们可以通过

$$f((a, b)) := g(a)(b).$$

⁶上面关于宇宙的描述是一个例外

⁷也称为 β -规约

⁸也称为 η -展开

来定义一个函数 $f : A \times B \rightarrow C$ 。我们在此避免使用 $g(a, b)$ 的写法, 以强调 g 不是乘积上的函数。(然而, 在本书后续部分, 我们经常将乘积上的函数与双变量的柯里化函数均写作 $g(a, b)$ 。) 这个定义方程是乘积类型的计算规则。

注意, 在集合论中, 我们会通过 $A \times B$ 的每个元素都是有序对这一事实来证明上述 f 的定义是合理的, 因此只需在这种有序对上定义 f 就足够了。与之相反, 类型论中情况是颠倒的: 我们假设一旦指定了函数在有序对上的值, $A \times B$ 上的函数就是良定义的, 并且(或者更准确地说, 由其对于依值函数的更一般版本, 见下文) 我们将因此能够证明 $A \times B$ 的每个元素都是一个有序对。从范畴论的角度来看, 我们可以说我们将乘积 $A \times B$ 定义为我们已经引入的“指数” $B \rightarrow C$ 的左伴随。

例如, 我们可以推导出**投影函数**:

$$\begin{aligned} \text{pr}_1 &: A \times B \rightarrow A \\ \text{pr}_2 &: A \times B \rightarrow B \end{aligned}$$

其定义方程为:

$$\begin{aligned} \text{pr}_1((a, b)) &\equiv a \\ \text{pr}_2((a, b)) &\equiv b. \end{aligned}$$

与其在每次定义函数时都调用这样一个函数定义原则, 一个替代方法是在一般情况下调用该原则, 然后在所有其他情况下简单地应用所得函数。也就是说, 我们会定义这样一种函数:

$$\text{rec}_{A \times B} : \prod_{C: \mathcal{U}} (A \rightarrow B \rightarrow C) \rightarrow A \times B \rightarrow C \quad (1.5.2)$$

其定义方程为:

$$\text{rec}_{A \times B}(C, g, (a, b)) \equiv g(a)(b).$$

然后, 我们可以定义

$$\begin{aligned} \text{pr}_1 &\equiv \text{rec}_{A \times B}(A, \lambda a. \lambda b. a) \\ \text{pr}_2 &\equiv \text{rec}_{A \times B}(B, \lambda a. \lambda b. b). \end{aligned}$$

而不是直接通过定义方程定义诸如 pr_1 和 pr_2 这样的函数。

我们将函数 $\text{rec}_{A \times B}$ 称为乘积类型的**递归子**。不幸的是, 因为没有任何递归发生, “递归子”的名称在此不太合适。其原因是, 在归纳类型的一般框架下, 乘积类型是一个退化的例子, 而对于诸如自然数这样的类型, 递归子确实是递归的。我们也可以谈论**Cartesian积的递归原则**, 其含义为我们可以通过给出函数在有序对上的值来定义如上所述的函数 $f : A \times B \rightarrow C$ 。

关于如何从投影函数中推导出递归子且反之亦然, 我们将其留给读者作为一个简单的练习。

对于单位类型, 我们也有一个递归子:

$$\text{rec}_1 : \prod_{C: \mathcal{U}} C \rightarrow \mathbf{1} \rightarrow C$$

其定义方程为

$$\text{rec}_1(C, c, \star) \equiv c.$$

尽管我们包含它是为了保持类型定义的模式, 但 $\mathbf{1}$ 的递归子完全无用, 因为我们可以通过简单地忽略类型为 $\mathbf{1}$ 的参数来直接定义这样的函数。

为了能够定义乘积类型上的依值函数, 我们必须推广递归子。给定 $C : A \times B \rightarrow \mathcal{U}$, 我们可以通过提供一个函数 $g : \prod_{(x:A)} \prod_{(y:B)} C((x, y))$ 来定义一个函数 $f : \prod_{(x:A \times B)} C(x)$, 其定义方程为

$$f((x, y)) \equiv g(x)(y).$$

例如, 通过这种方式我们可以证明命题性唯一性原理, 该原理表明 $A \times B$ 的每个元素都等于一个有序对。具体地, 我们可以构造一个函数

$$\text{uniq}_{A \times B} : \prod_{x:A \times B} ((\text{pr}_1(x), \text{pr}_2(x)) =_{A \times B} x).$$

这里我们使用恒等类型, 我们将在下面的§1.12中介绍它。然而, 我们现在只需要知道对于任意 $x : A$, 存在一个自反元 $\text{refl}_x : x =_A x$ 。由这一点, 我们可以定义

$$\text{uniq}_{A \times B}((a, b)) \equiv \text{refl}_{(a, b)}.$$

这个构造是可行的, 因为在 $x \equiv (a, b)$ 的情况下, 我们可以使用投影的定义方程计算

$$(\text{pr}_1((a, b)), \text{pr}_2((a, b))) \equiv (a, b)$$

因此,

$$\text{refl}_{(a, b)} : (\text{pr}_1((a, b)), \text{pr}_2((a, b))) = (a, b)$$

是良类型的, 因为等式的两边是判断相等的。

更一般地, 以这种方式定义依值函数的能力意味着, 要证明乘积类型所有元素的某个性质, 只需对其典范元素(即有序对)证明该性质即可。当我们遇到归纳类型(如自然数)时, 类似的性质将使我们能够通过归纳法书写证明。因此, 如果我们如前所述在通用情况下应用此原理, 所得函数被称为乘积类型的归纳法: 给定 $A, B : \mathcal{U}$, 我们有

$$\text{ind}_{A \times B} : \prod_{C:A \times B \rightarrow \mathcal{U}} \left(\prod_{(x:A)} \prod_{(y:B)} C((x, y)) \right) \rightarrow \prod_{x:A \times B} C(x)$$

其定义方程为

$$\text{ind}_{A \times B}(C, g, (a, b)) \equiv g(a)(b).$$

类似地, 我们可以说定义在有序对上的依值函数可以通过Cartesian积的归纳原理得到。容易看出, 递归子只是归纳法在 C 为常类型族时的特例。因为归纳法描述了如何使用乘积类型的元素, 所以归纳法也称为(依值)消去子, 而递归规则称为非依值消去子。

单位类型的归纳法比递归子更有用:

$$\text{ind}_1 : \prod_{C:1 \rightarrow \mathcal{U}} C(\star) \rightarrow \prod_{x:1} C(x)$$

其定义方程为

$$\text{ind}_1(C, c, \star) \equiv c.$$

归纳法使我们能够证明 **1** 的命题性唯一性原理, 该原理断言其唯一的居留元是 $\star$ 。也就是说, 我们可以通过使用定义方程

$$\text{uniq}_1(\star) \equiv \text{refl}_\star$$

或者等价地使用归纳法来构造

$$\text{uniq}_1 : \prod_{x:1} x = \star$$

$$\text{uniq}_1 \equiv \text{ind}_1(\lambda x. x = \star, \text{refl}_\star).$$

1.6 依值对类型 (Σ -类型)

正如我们将函数类型(第1.2节)推广为依值函数类型(第1.4节)一样, 将第1.5节中的积类型推广, 以允许有序对的第二个分量的类型随第一个分量的选择而变化, 这在很多情况下是有用的。这被称为**依值对类型**, 或 **Σ -类型**, 因为在集合论中, 它对应于给定类型上的索引和(即余积或不相交并)。

给定一个类型 $A : \mathcal{U}$ 和一个类型族 $B : A \rightarrow \mathcal{U}$, 依值对类型写作 $\sum_{(x:A)} B(x) : \mathcal{U}$ 。替代记号有

$$\sum_{(x:A)} B(x) \qquad \sum_{(x:A)} B(x) \qquad \Sigma(x : A), B(x).$$

与其他绑定构造(如 λ -抽象和 Π 类型)一样, Σ 自动作用于表达式的其余部分, 除非被分隔, 因此例如 $\sum_{(x:A)} B(x) \rightarrow C$ 表示 $\sum_{(x:A)} (B(x) \rightarrow C)$ 。

构造依值对类型元素的方式是通过配对: 给定 $a : A$ 和 $b : B(a)$, 我们有 $(a, b) : \sum_{(x:A)} B(x)$ 。如果 B 是常类型族, 那么依值对类型就是普通的Cartesian积类型:

$$\left(\sum_{x:A} B \right) \equiv (A \times B).$$

Σ -类型上的所有构造都是积类型上对应构造的直接推广, 其中非依值函数常常被依值函数所取代。

例如, 递归原理指出 通过提供一个函数 $g : \prod_{(x:A)} B(x) \rightarrow C$, 我们可以定义一个从 Σ -类型出发的非依值函数 $f : (\sum_{(x:A)} B(x)) \rightarrow C$, 再通过定义方程

$$f((a, b)) \equiv g(a)(b).$$

来定义 f 。

例如, 我们可以从 Σ -类型推导出第一投影:

$$\text{pr}_1 : \left(\sum_{x:A} B(x) \right) \rightarrow A$$

其定义方程为

$$\text{pr}_1((a, b)) \equiv a.$$

然而, 由于有序对 $(a, b) : \sum_{(x:A)} B(x)$ 的第二个分量的类型是 $B(a)$, 第二投影必须是一个依值函数, 其类型依赖于第一投影函数:

$$\text{pr}_2 : \prod_{p : \sum_{(x:A)} B(x)} B(\text{pr}_1(p)).$$

因此我们需要 Σ -类型的归纳原理 (即“依值消去器”)。该原理指出, 给定一个函数

$$g : \prod_{(a:A)} \prod_{(b:B(a))} C((a, b)).$$

可以构造一个从 Σ -类型到族 $C : (\sum_{(x:A)} B(x)) \rightarrow \mathcal{U}$ 的依值函数

$$f : \prod_{p : \sum_{(x:A)} B(x)} C(p)$$

其定义方程为

$$f((a, b)) \equiv g(a)(b).$$

将其应用于 $C(p) := B(\text{pr}_1(p))$, 我们可以定义 $\text{pr}_2 : \prod_{(p:\sum_{(x:A)} B(x))} B(\text{pr}_1(p))$ 显然其方程为

$$\text{pr}_2((a, b)) := b.$$

为了证明这是正确的, 我们注意到 $B(\text{pr}_1((a, b))) \equiv B(a)$ 。应用了 pr_1 的定义方程后, 并且确实有 $b : B(a)$ 。

我们可以将递归原理和归纳原理打包到 Σ 的递归子中:

$$\text{rec}_{\sum_{(x:A)} B(x)} : \prod_{(C:\mathcal{U})} \left(\prod_{(x:A)} B(x) \rightarrow C \right) \rightarrow \left(\sum_{(x:A)} B(x) \right) \rightarrow C$$

其定义方程为

$$\text{rec}_{\sum_{(x:A)} B(x)}(C, g, (a, b)) := g(a)(b)$$

以及相应的归纳算子:

$$\text{ind}_{\sum_{(x:A)} B(x)} : \prod_{(C:\sum_{(x:A)} B(x) \rightarrow \mathcal{U})} \left(\prod_{(a:A)} \prod_{(b:B(a))} C((a, b)) \right) \rightarrow \prod_{(p:\sum_{(x:A)} B(x))} C(p)$$

其定义方程为

$$\text{ind}_{\sum_{(x:A)} B(x)}(C, g, (a, b)) := g(a)(b).$$

如前所述, 递归子是归纳法在 C 为常类型族时的特例。

更进一步地, 考虑以下原理作为例子, 其中 A 和 B 是类型且 $R : A \rightarrow B \rightarrow \mathcal{U}$:

$$\text{ac} : \left(\prod_{(x:A)} \sum_{(y:B)} R(x, y) \right) \rightarrow \left(\sum_{(f:A \rightarrow B)} \prod_{(x:A)} R(x, f(x)) \right).$$

我们可以将 R 视为 A 和 B 之间的一个“证明相关关系”, 其中 $R(a, b)$ 是 $a : A$ 和 $b : B$ 相关性的见证类型。那么 ac 直观上表示, 如果我们有一个依值函数 g , 其为每个 $a : A$ 分配一个依值对 (b, r) , 其中 $b : B$ 且 $r : R(a, b)$, 那么我们就有一个函数 $f : A \rightarrow B$ 和一个依值函数, 其为每个 $a : A$ 分配一个 $R(a, f(a))$ 的见证。我们的直觉告诉我们, 我们可以直接将 g 的值拆分为其分量。实际上, 使用我们刚刚定义的投影函数, 我们可以定义:

$$\text{ac}(g) := (\lambda x. \text{pr}_1(g(x)), \lambda x. \text{pr}_2(g(x))).$$

为了验证这是良类型的, 注意如果 $g : \prod_{(x:A)} \sum_{(y:B)} R(x, y)$, 我们有

$$\begin{aligned} \lambda x. \text{pr}_1(g(x)) &: A \rightarrow B, \\ \lambda x. \text{pr}_2(g(x)) &: \prod_{(x:A)} R(x, \text{pr}_1(g(x))). \end{aligned}$$

此外, 对 ac 陪域中正在求和的类型族 $\lambda f. \prod_{(x:A)} R(x, f(x))$, 将其应用于函数 $\lambda x. \text{pr}_1(g(x))$, 会得到类型 $\prod_{(x:A)} R(x, \text{pr}_1(g(x)))$:

$$\prod_{(x:A)} R(x, \text{pr}_1(g(x))) \equiv (\lambda f. \prod_{(x:A)} R(x, f(x))) (\lambda x. \text{pr}_1(g(x))).$$

因此, 我们有

$$(\lambda x. \text{pr}_1(g(x)), \lambda x. \text{pr}_2(g(x))) : \sum_{(f:A \rightarrow B)} \prod_{(x:A)} R(x, f(x))$$

这正是我们所需要的

如果我们将 Π 读作“对于所有”而将 Σ 读作“存在”, 那么函数 ac 的类型表达的是: 如果对于所有 $x : A$ 存在 $y : B$ 使得 $R(x, y)$, 那么存在函数 $f : A \rightarrow B$ 使得对于所有 $x : A$ 我们有 $R(x, f(x))$ 。

由于这听起来像是选择公理的一个版本, 函数 ac 传统上被称为**类型论的选择公理**, 并且正如我们刚才所示, 它可以直接从类型论的规则证明, 而不必作为公理引入。然而, 注意实际上并不涉及选择, 因为选择已经在前提中给予我们了: 我们所要做的只是将其拆分为两个函数: 一个代表选择, 另一个代表其正确性。在§3.8中, 我们将给出另一个“选择公理”的表述, 它更接近通常所提的形式。

依值对类型经常用于定义数学结构的类型, 这些结构通常由几个依值的数据片段组成。举一个简单的例子, 假设我们想把一个**原群**定义为一个类型 A 连同其上的二元运算 $m : A \rightarrow A \rightarrow A$ 。短语“连同”(以及同义的“配备上”)的准确含义是“一个原群”是一个由类型 $A : \mathcal{U}$ 和运算 $m : A \rightarrow A \rightarrow A$ 组成的对 (A, m) 。由于这对的第二分量 m 的类型 $A \rightarrow A \rightarrow A$ 依赖于其第一分量 A , 这样的对属于一个依值对类型。因此, 定义“一个原群是一个类型 A 连同其上二元运算 $m : A \rightarrow A \rightarrow A$ ”应理解为将原群的类型定义为

$$\text{Magma} := \sum_{A:\mathcal{U}} (A \rightarrow A \rightarrow A).$$

给定一个原群, 我们用第一投影 pr_1 提取其底层类型(其“载体”), 用第二投影 pr_2 提取其运算。当然, 如果结构由多于两个数据片段构建, 就需要迭代出现的对类型, 它们可能只是部分依值的; 例如, 带点原群(配备有基点 $e : A$ 的原群 (A, m))的类型是

$$\text{PointedMagma} := \sum_{A:\mathcal{U}} (A \rightarrow A \rightarrow A) \times A.$$

我们通常还希望对此类结构施加公理, 例如使一个带点原群成为么半群或群。这也可以使用 Σ -类型来完成; 参见§1.11。

在本书的其余部分, 我们有时会明确给出此类定义, 但最终我们相信读者能够将它们从英语(中文)翻译成 Σ -类型。我们通常也遵循常见的数学实践, 对此类结构及其载体使用相同的字母(这相当于在记号中隐去适当的投影函数): 也就是说, 我们将谈论一个原群 A 及其运算 $m : A \rightarrow A \rightarrow A$ 。

注意, PointedMagma 的典范元素形如 $(A, (m, e))$, 其中 $A : \mathcal{U}$, $m : A \rightarrow A \rightarrow A$, 且 $e : A$ 。由于此类迭代 Σ -类型频繁出现, 我们使用通常的有序三元组、四元组等记号来表示向右结合的嵌套对(可能是依值的)。也就是说, 我们有 $(x, y, z) := (x, (y, z))$ 和 $(x, y, z, w) := (x, (y, (z, w)))$, 等等。

1.7 余积类型

给定 $A, B : \mathcal{U}$, 我们引入它们的**余积类型** $A + B : \mathcal{U}$ 。这在集合论中对应于无交并, 我们也可以使用该名称。在类型论中, 与函数和乘积的情况一样, 由于之前没有给出“类型并”的概念, 所以余积必须是一个基本构造。我们还为其引入一个单位元: **空类型** $0 : \mathcal{U}$ 。

构造 $A + B$ 元素有两种方式, 对于 $a : A$ 来说是 $\text{inl}(a) : A + B$, 对于 $b : B$ 来说是 $\text{inr}(b) : A + B$ 。(名称 inl 和 inr 是“左注入”和“右注入”的缩写。) 没有构造空类型元素的方法。

要构造一个非依值函数 $f : A + B \rightarrow C$, 我们需要函数 $g_0 : A \rightarrow C$ 和 $g_1 : B \rightarrow C$ 。并通过定义方程

$$\begin{aligned} f(\text{inl}(a)) &:= g_0(a), \\ f(\text{inr}(b)) &:= g_1(b). \end{aligned}$$

来构造 f 。也就是说, 函数 f 是通过**分类讨论**来定义的。如前所述, 我们可以推导出递归子:

$$\text{rec}_{A+B} : \prod_{(C:\mathcal{U})} (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow A + B \rightarrow C$$

其定义方程为

$$\begin{aligned}\text{rec}_{A+B}(C, g_0, g_1, \text{inl}(a)) &\equiv g_0(a), \\ \text{rec}_{A+B}(C, g_0, g_1, \text{inr}(b)) &\equiv g_1(b).\end{aligned}$$

我们总是可以构造一个函数 $f : \mathbf{0} \rightarrow C$ 而无需给出任何定义方程, 因为 $\mathbf{0}$ 中没有元素来定义 f 。因此, $\mathbf{0}$ 的递归子是

$$\text{rec}_0 : \prod_{(C:\mathcal{U})} \mathbf{0} \rightarrow C,$$

它构造了从空类型到任何其他类型的典范函数。在逻辑上, 它对应于原则 由空推全部。

要构造一个从余积出发的依值函数 $f : \prod_{(x:A+B)} C(x)$, 我们假设给定类型族 $C : (A+B) \rightarrow \mathcal{U}$, 并要求

$$\begin{aligned}g_0 &: \prod_{a:A} C(\text{inl}(a)), \\ g_1 &: \prod_{b:B} C(\text{inr}(b)).\end{aligned}$$

从而得到 f 及其定义方程:

$$\begin{aligned}f(\text{inl}(a)) &\equiv g_0(a), \\ f(\text{inr}(b)) &\equiv g_1(b).\end{aligned}$$

我们将此方案打包成余积的归纳法:

$$\text{ind}_{A+B} : \prod_{(C:(A+B) \rightarrow \mathcal{U})} \left(\prod_{(a:A)} C(\text{inl}(a)) \right) \rightarrow \left(\prod_{(b:B)} C(\text{inr}(b)) \right) \rightarrow \prod_{(x:A+B)} C(x).$$

如前所述, 当类型族 C 为常数时, 递归子便会出现。

空类型的归纳原理

$$\text{ind}_0 : \prod_{(C:\mathbf{0} \rightarrow \mathcal{U})} \prod_{(z:\mathbf{0})} C(z)$$

给出一种从空类型定义平凡依值函数的方法。

1.8 布尔类型

布尔类型 $\mathbf{2} : \mathcal{U}$ 被设计为恰好有两个元素 $0_2, 1_2 : \mathbf{2}$ 。显然, 我们可以从余积类型 和单位类型中将它构造为 $\mathbf{1} + \mathbf{1}$ 。然而, 由于我们经常使用这一类型, 我们在此给出显式的规则。实际上, 我们将观察到我们也可以反过来从 Σ -类型和 $\mathbf{2}$ 中推导出二元余积。

要导出一个函数 $f : \mathbf{2} \rightarrow C$, 我们需要 $c_0, c_1 : C$ 并附加一个定义方程

$$\begin{aligned}f(0_2) &\equiv c_0, \\ f(1_2) &\equiv c_1.\end{aligned}$$

该递归子对应于函数式编程中的 if-then-else 构造:

$$\text{rec}_2 : \prod_{C:\mathcal{U}} C \rightarrow C \rightarrow \mathbf{2} \rightarrow C$$

其定义方程为

$$\begin{aligned}\text{rec}_2(C, c_0, c_1, 0_2) &\equiv c_0, \\ \text{rec}_2(C, c_0, c_1, 1_2) &\equiv c_1.\end{aligned}$$

给定 $C : \mathbf{2} \rightarrow \mathcal{U}$, 要导出一个依值函数 $f : \prod_{(x:\mathbf{2})} C(x)$, 我们需要 $c_0 : C(0_2)$ 和 $c_1 : C(1_2)$, 在这种情况下我们可以给出定义方程

$$\begin{aligned} f(0_2) &::= c_0, \\ f(1_2) &::= c_1. \end{aligned}$$

我们将其打包成归纳原理

$$\text{ind}_2 : \prod_{(C:\mathbf{2} \rightarrow \mathcal{U})} C(0_2) \rightarrow C(1_2) \rightarrow \prod_{(x:\mathbf{2})} C(x)$$

其定义方程为

$$\begin{aligned} \text{ind}_2(C, c_0, c_1, 0_2) &::= c_0, \\ \text{ind}_2(C, c_0, c_1, 1_2) &::= c_1. \end{aligned}$$

例如, 通过归纳法, 我们可以推断出, 正如期望的那样, $\mathbf{2}$ 的每个元素要么是 1_2 要么是 0_2 。如前所述, 为了陈述这一点, 我们使用了尚未引入的相等类型, 但我们只需要每个东西都通过 $\text{refl}_x : x = x$ 与自身相等这一事实。因此, 我们构造一个

$$\prod_{x:\mathbf{2}} (x = 0_2) + (x = 1_2), \quad (1.8.1)$$

的元素, 即一个函数, 为每个 $x : \mathbf{2}$ 分配一个等式 $x = 0_2$ 或一个等式 $x = 1_2$ 。我们使用 $\mathbf{2}$ 的归纳原理定义这个元素, 其中 $C(x) ::= (x = 0_2) + (x = 1_2)$; 两个输入是 $\text{inl}(\text{refl}_{0_2}) : C(0_2)$ 和 $\text{inr}(\text{refl}_{1_2}) : C(1_2)$ 。换句话说, 我们的 (1.8.1) 的元素是

$$\text{ind}_2(\lambda x. (x = 0_2) + (x = 1_2), \text{inl}(\text{refl}_{0_2}), \text{inr}(\text{refl}_{1_2})).$$

我们曾指出 Σ -类型可以视为类似于索引不交并, 而余积是二元不交并。自然地, 我们期望二元不交并 $A + B$ 可以构造为在二元类型 $\mathbf{2}$ 上的索引不交并。为此我们需要一个类型族 $P : \mathbf{2} \rightarrow \mathcal{U}$ 使得 $P(0_2) \equiv A$ 且 $P(1_2) \equiv B$ 。实际上, 我们可以通过 $\mathbf{2}$ 的递归原理精确地得到这样的族。(通过利用宇宙 $\mathcal{U}$ 本身是一个类型这一事实, 从而归纳和递归定义类型族, 是类型论的一个微妙而重要的能力。) 因此, 我们可以定义

$$A + B ::= \sum_{x:\mathbf{2}} \text{rec}_2(\mathcal{U}, A, B, x)$$

其中

$$\begin{aligned} \text{inl}(a) &::= (0_2, a), \\ \text{inr}(b) &::= (1_2, b). \end{aligned}$$

我们将其留作练习, 以从这个定义推导出余积类型的归纳原理。(另见练习1.5和第5.2节。)

我们可以将同样的想法应用于乘积和 Π -类型: 我们可以定义

$$A \times B ::= \prod_{x:\mathbf{2}} \text{rec}_2(\mathcal{U}, A, B, x).$$

然后可以使用 $\mathbf{2}$ 的归纳构造对:

$$(a, b) ::= \text{ind}_2(\text{rec}_2(\mathcal{U}, A, B), a, b)$$

而投影是对其直接的应用

$$\begin{aligned}\text{pr}_1(p) &:= p(0_2), \\ \text{pr}_2(p) &:= p(1_2).\end{aligned}$$

以这种方式定义的二元积的归纳原理的推导稍微复杂一些, 并且需要函数外延性, 我们将在第2.9节介绍它。此外, 我们得不到相同的判断相等性; 见练习1.6。这是将一种类型编码为另一种类型时经常出现的问题; 我们将在第5.5节回到这个问题。

我们可能偶尔将 $\mathbf{2}$ 的元素 0_2 和 1_2 分别称为“假”和“真”。但请注意, 与经典数学不同, 我们不使用 $\mathbf{2}$ 的元素作为真值 或作为命题。(相反, 我们将命题与类型等同; 见第1.11节。) 特别地, 类型 $A \rightarrow \mathbf{2}$ 通常不是 A 的幂集; 它仅表示 A 的“可判定”子集(见第3章)。

1.9 自然数

到目前为止, 我们拥有通过抽象运算构造新类型的规则, 但为了进行具体的数学研究, 我们还需要一些具体的类型, 例如数的类型。其中最基础的是自然数的类型 $\mathbb{N} : \mathcal{U}$; 一旦有了该类型, 我们就可以构造整数、有理数、实数等等(见第11章)。

$\mathbb{N}$ 的元素使用 $0 : \mathbb{N}$ 和后继运算 $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$ 构造。在表示自然数时, 我们采用通常的十进制记法 $1 := \text{succ}(0)$, $2 := \text{succ}(1)$, $3 := \text{succ}(2)$,

自然数的基本性质是我们可以通过递归定义函数并通过归纳进行证明 — 此时“递归”和“归纳”这两个词具有更熟悉的含义。要通过递归的方法从自然数构造一个非依值函数 $f : \mathbb{N} \rightarrow C$, 只需提供一个起点 $c_0 : C$ 和一个“下一步”函数 $c_s : \mathbb{N} \rightarrow C \rightarrow C$ 。从而通过下列定义方程构造 f

$$\begin{aligned}f(0) &:= c_0, \\ f(\text{succ}(n)) &:= c_s(n, f(n)).\end{aligned}$$

此时我们说, f 是通过 **原始递归** 定义的。

例如, 让我们看看如何定义一个将参数翻倍的自然数函数。在这种情况下, 我们有 $C := \mathbb{N}$ 。首先需要提供 $\text{double}(0)$ 的值, 这很简单: 令 $c_0 := 0$ 。接下来, 对自然数 n , 要计算 $\text{double}(\text{succ}(n))$, 我们首先计算 $\text{double}(n)$, 再执行两次后继运算。这由递推式 $c_s(n, y) := \text{succ}(\text{succ}(y))$ 捕获。注意 c_s 的第二个参数 y 代表递归调用 $\text{double}(n)$ 的结果。

因此, 通过这种原始递归定义的方式来定义 $\text{double} : \mathbb{N} \rightarrow \mathbb{N}$, 可以得到定义方程:

$$\begin{aligned}\text{double}(0) &:= 0 \\ \text{double}(\text{succ}(n)) &:= \text{succ}(\text{succ}(\text{double}(n))).\end{aligned}$$

其计算行为确实正确: 例如, 我们有

$$\begin{aligned}\text{double}(2) &\equiv \text{double}(\text{succ}(\text{succ}(0))) \\ &\equiv c_s(\text{succ}(0), \text{double}(\text{succ}(0))) \\ &\equiv \text{succ}(\text{succ}(\text{double}(\text{succ}(0)))) \\ &\equiv \text{succ}(\text{succ}(c_s(0, \text{double}(0)))) \\ &\equiv \text{succ}(\text{succ}(\text{succ}(\text{succ}(\text{double}(0))))) \\ &\equiv \text{succ}(\text{succ}(\text{succ}(\text{succ}(c_0)))) \\ &\equiv \text{succ}(\text{succ}(\text{succ}(\text{succ}(0)))) \\ &\equiv 4.\end{aligned}$$

我们也可以通过柯里化并允许 C 是函数类型, 用原始递归定义多变量函数。例如, 我们可以定义加法 $\text{add} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, 其中 $C \equiv \mathbb{N} \rightarrow \mathbb{N}$ 且具有以下“起点”和“下一步”数据:

$$\begin{aligned} c_0 &: \mathbb{N} \rightarrow \mathbb{N} \\ c_0(n) &\equiv n \\ c_s &: \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}) \\ c_s(m, g)(n) &\equiv \text{succ}(g(n)). \end{aligned}$$

由此我们得到 $\text{add} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 满足定义等式

$$\begin{aligned} \text{add}(0, n) &\equiv n \\ \text{add}(\text{succ}(m), n) &\equiv \text{succ}(\text{add}(m, n)). \end{aligned}$$

照常, 我们将 $\text{add}(m, n)$ 写作 $m + n$ 。邀请读者验证 $2 + 2 \equiv 4$ 。

与之前的情况一样, 我们可以将原始递归法打包成一个递归子:

$$\text{rec}_{\mathbb{N}} : \prod_{(C:\mathcal{U})} C \rightarrow (\mathbb{N} \rightarrow C \rightarrow C) \rightarrow \mathbb{N} \rightarrow C$$

其定义方程为

$$\begin{aligned} \text{rec}_{\mathbb{N}}(C, c_0, c_s, 0) &\equiv c_0, \\ \text{rec}_{\mathbb{N}}(C, c_0, c_s, \text{succ}(n)) &\equiv c_s(n, \text{rec}_{\mathbb{N}}(C, c_0, c_s, n)). \end{aligned}$$

使用 $\text{rec}_{\mathbb{N}}$ 我们可以如下表示 double 和 add :

$$\text{double} \equiv \text{rec}_{\mathbb{N}}(\mathbb{N}, 0, \lambda n. \lambda y. \text{succ}(\text{succ}(y))) \quad (1.9.1)$$

$$\text{add} \equiv \text{rec}_{\mathbb{N}}(\mathbb{N} \rightarrow \mathbb{N}, \lambda n. n, \lambda m. \lambda g. \lambda n. \text{succ}(g(n))). \quad (1.9.2)$$

当然, 仅使用原始递归法可定义的所有函数都将是可计算的。(然而, 高阶函数类型的存在 — 即以其他函数为参数的函数 — 意味着我们可以定义比通常的原始递归函数更多的函数; 例如参见练习1.10。)这在构造性数学中是合适的; 在第3.4节和第3.8节, 我们将看到如何扩充类型论以便定义更一般的数学函数。

我们现在遵循与其他类型相同的方法, 将原始递归推广到依值函数以获得一个归纳原理。因此, 假设给定一个类型族 $C : \mathbb{N} \rightarrow \mathcal{U}$, 一个元素 $c_0 : C(0)$, 和一个函数 $c_s : \prod_{(n:\mathbb{N})} C(n) \rightarrow C(\text{succ}(n))$; 那么我们可以构造 $f : \prod_{(n:\mathbb{N})} C(n)$ 其定义方程为:

$$\begin{aligned} f(0) &\equiv c_0, \\ f(\text{succ}(n)) &\equiv c_s(n, f(n)). \end{aligned}$$

我们也可以将其打包成一个单一函数

$$\text{ind}_{\mathbb{N}} : \prod_{(C:\mathbb{N} \rightarrow \mathcal{U})} C(0) \rightarrow \left(\prod_{(n:\mathbb{N})} C(n) \rightarrow C(\text{succ}(n)) \right) \rightarrow \prod_{(n:\mathbb{N})} C(n)$$

其定义方程为

$$\begin{aligned} \text{ind}_{\mathbb{N}}(C, c_0, c_s, 0) &\equiv c_0, \\ \text{ind}_{\mathbb{N}}(C, c_0, c_s, \text{succ}(n)) &\equiv c_s(n, \text{ind}_{\mathbb{N}}(C, c_0, c_s, n)). \end{aligned}$$

在这里我们终于看到了与经典的归纳证明概念的联系。回忆在类型论中, 我们用类型表示命题, 并通过居留相应的类型来证明命题。特别地, 自然数的性质由一个类型族 $P : \mathbb{N} \rightarrow \mathcal{U}$ 表示。从这个观点来看, 上述归纳原理是说, 如果我们能证明 $P(0)$, 并且对于任何 n , 在假设 $P(n)$ 的情况下我们能证明 $P(\text{succ}(n))$, 那么我们对所有 n 都有 $P(n)$ 。当然, 这正是通常的数学归纳法。

例如, 考虑我们如何表示 $+$ 满足结合律的一个显式证明。(我们实际上不会以这种风格写出证明, 但它作为一个有用的例子来理解归纳在类型论中如何形式化表示。) 要推导

$$\text{assoc} : \prod_{i,j,k:\mathbb{N}} i + (j + k) = (i + j) + k,$$

只需提供

$$\text{assoc}_0 : \prod_{j,k:\mathbb{N}} 0 + (j + k) = (0 + j) + k$$

和

$$\text{assoc}_s : \prod_{i:\mathbb{N}} \left(\prod_{j,k:\mathbb{N}} i + (j + k) = (i + j) + k \right) \rightarrow \prod_{j,k:\mathbb{N}} \text{succ}(i) + (j + k) = (\text{succ}(i) + j) + k.$$

要推导 assoc_0 , 回想起 $0 + n \equiv n$, 因此 $0 + (j + k) \equiv j + k \equiv (0 + j) + k$ 。因此我们可以直接设

$$\text{assoc}_0(j, k) := \text{refl}_{j+k}.$$

对于 assoc_s , 回想起 $+$ 的定义给出 $\text{succ}(m) + n \equiv \text{succ}(m + n)$, 因此

$$\begin{aligned} \text{succ}(i) + (j + k) &\equiv \text{succ}(i + (j + k)) && \text{且} \\ (\text{succ}(i) + j) + k &\equiv \text{succ}((i + j) + k). \end{aligned}$$

因此, assoc_s 的输出类型等价于 $\text{succ}(i + (j + k)) = \text{succ}((i + j) + k)$ 。但其输入(“归纳假设”)产生 $i + (j + k) = (i + j) + k$, 因此只需援引以下事实: 如果两个自然数相等, 那么它们的后继也相等。(我们将在引理2.2.1中使用恒等类型的归纳法证明这个明显的事实。) 我们称后一个事实为 $\text{ap}_{\text{succ}} : (m =_{\mathbb{N}} n) \rightarrow (\text{succ}(m) =_{\mathbb{N}} \text{succ}(n))$, 因此我们可以定义

$$\text{assoc}_s(i, h, j, k) := \text{ap}_{\text{succ}}(h(j, k)).$$

将这些与 $\text{ind}_{\mathbb{N}}$ 放在一起, 我们得到结合性的一个证明。

1.10 模式匹配与递归

自然数带来了出乎迄今为止所考虑的所有类型的一个微妙特性。例如, 考虑余积类型, 我们可以用递归子定义一个函数 $f : A + B \rightarrow C$:

$$f := \text{rec}_{A+B}(C, g_0, g_1)$$

或者通过给出定义方程:

$$\begin{aligned} f(\text{inl}(a)) &:= g_0(a) \\ f(\text{inr}(b)) &:= g_1(b). \end{aligned}$$

要从 f 的前一种表达式得到后一种, 我们只需使用递归子的计算规则。相反地, 给定任何定义方程

$$\begin{aligned} f(\text{inl}(a)) &:= \Phi_0 \\ f(\text{inr}(b)) &:= \Phi_1 \end{aligned}$$

其中 Φ_0 和 Φ_1 是可能分别涉及变量 a 和 b 的表达式, 我们可以通过使用 λ -抽象 等价地用递归子表达这些方程:

$$f := \text{rec}_{A+B}(C, \lambda a. \Phi_0, \lambda b. \Phi_1).$$

然而, 在自然数的情况下, 对诸如 `double` 的函数的“定义方程”:

$$\text{double}(0) := 0 \quad (1.10.1)$$

$$\text{double}(\text{succ}(n)) := \text{succ}(\text{succ}(\text{double}(n))) \quad (1.10.2)$$

在右侧会涉及函数 `double` 本身。然而, 我们仍然希望能够给出这些方程, 而不是 (1.9.1), 作为 `double` 的定义, 因为它们更方便且更易读。解决方案是将 (1.10.2) 右侧的表达式“`double(n)`”视为递归调用结果的占位符, 在形式为 $\text{double} := \text{rec}_{\mathbb{N}}(\mathbb{N}, c_0, c_s)$ 的定义中, 该结果将是 c_s 的第二个参数。

更一般地, 如果我们有一个函数 $f : \mathbb{N} \rightarrow C$ 的“定义”, 例如

$$f(0) := \Phi_0$$

$$f(\text{succ}(n)) := \Phi_s$$

其中 Φ_0 是类型 C 的一个表达式, 而 Φ_s 是类型 C 的一个表达式, 它可能涉及变量 n 以及符号“ $f(n)$ ”, 我们可以将其转换为定义

$$f := \text{rec}_{\mathbb{N}}(C, \Phi_0, \lambda n. \lambda r. \Phi'_s)$$

其中 Φ'_s 是通过将“ $f(n)$ ”的所有出现替换为新变量 r 从 Φ_s 得到的。

这种通过递归定义函数(或者更一般地, 通过归纳定义依值函数)的风格非常方便, 因此我们经常采用它。它被称为通过 **模式匹配** 定义。当然, 它非常类似于计算机程序员如何定义一个递归函数, 其函数体字面上包含对自身的递归调用。然而, 与程序员不同, 在进行何种递归调用时, 我们会在这一方面受到限制: 为了使这样的定义能够使用递归原理重新表达, 正在定义的函数 f 在 $f(\text{succ}(n))$ 的函数体中只能作为复合符号“ $f(n)$ ”的一部分出现。否则, 我们可能会写出无意义的函数, 例如

$$f(0) := 0$$

$$f(\text{succ}(n)) := f(\text{succ}(\text{succ}(n))).$$

如果程序员写出这样的函数, 它将在任何正输入上永远调用自身, 进入无限循环且永不返回值。然而在数学中, 要配得上函数这个名字, 必须始终为每个输入值关联一个唯一的输出值, 因此这是不可接受的。

当我们引入更复杂的归纳类型时(见第5章、第6章和第11章), 这一点将更加重要。每当我们引入一种新的归纳定义时, 我们总是首先推导其归纳原理。只有这样, 我们才引入一种适当的“模式匹配”, 它可以被证明是归纳法的简写。

1.11 命题即类型

如引言中所述, 在类型论中, 证明一个命题为真对应于展示与该命题对应的类型的一个元素。我们将该类型的元素视为该命题为真的证据或见证。(它们有时甚至被称为证明, 但这种术语可能引起误解, 因此我们通常避免使用它。)然而, 一般来说, 我们不会显式地构造见证; 相反, 我们将用普通的数学行文风格呈现证明, 使得它们可以被翻译成类型的元素。这与经典集合论中的推理没有什么不同, 在其中, 我们并不期望看到使用谓词逻辑规则和集合论公理的显式推导。

然而,在某些重要方面,类型论观点的证明仍有不同。类型论逻辑的基本原理是,一个命题不仅仅为真或假,而是可以被视为其真理性的所有可能见证的集合。在这种构想下,证明不仅仅是交流数学的手段,而且本身就是数学对象,与更熟悉的对象如数字、映射、群等并列。因此,由于类型对可用的数学对象进行分类并支配它们如何交互,命题无非是特殊的类型——即其元素是证明的类型。

使这种等同可行的基本观察是,我们有以下在命题上的逻辑运算(用英语表达)与其对应见证类型上的类型论运算之间的自然对应。

英语	类型论
真	1
假	0
A 且 B	$A \times B$
A 或 B	$A + B$
若 A 则 B	$A \rightarrow B$
A 当且仅当 B	$(A \rightarrow B) \times (B \rightarrow A)$
非 A	$A \rightarrow \mathbf{0}$

对应的要点在于,在每种情况下,构造和使用右侧类型元素的规则对应于推理左侧命题的规则。例如,证明形式为“ A 且 B ”的陈述的基本方法是证明 A 并且也证明 B ,而构造 $A \times B$ 元素的基本方式是作为一个对 (a, b) ,其中 a 是 A 的一个元素(或见证), b 是 B 的一个元素(或见证)。如果我们想使用“ A 且 B ”来证明其他东西,我们可以自由地在这样做时使用 A 和 B ,类似于 $A \times B$ 的归纳原理允许我们通过使用 A 和 B 的元素来构造一个从它出发的函数。

类似地,证明蕴涵“若 A 则 B ”的基本方法是假设 A 并证明 B ,而构造 $A \rightarrow B$ 元素的基本方式是给出一个表示 B 元素(见证)的表达式,该表达式可能涉及类型 A 的未指定变量元素(见证)。而使用蕴涵“若 A 则 B ”的基本方法是,如果我们知道 A 则推导出 B ,类似于我们可以将函数 $f : A \rightarrow B$ 应用于 A 的元素以产生 B 的元素。我们强烈鼓励读者做这个练习:验证支配其他类型构造子的规则是否合理地转化为逻辑。

特别值得注意的是,空类型 **0** 对应于假。在逻辑上说话时,我们将 **0** 的居留元称为**矛盾**:因此没有方法证明一个矛盾,⁹ 而从矛盾可以推导出任何东西。我们也将类型 A 的**否定** 定义为

$$\neg A := A \rightarrow \mathbf{0}.$$

因此, $\neg A$ 的一个见证是一个函数 $A \rightarrow \mathbf{0}$, 我们可以通过假设 $x : A$ 并推导出 **0** 的一个元素来构造它。注意,尽管如引言中所讨论的,我们得到的逻辑是“构造性的”,但这种“反证法”(假设 A 并推导出矛盾,从而断定 $\neg A$) 在构造性意义下是完全有效的:它只是援引了“否定”的含义。不被允许的那种“反证法”是通过假设 $\neg A$ 并推导出矛盾来证明 A 。在构造性意义下,这样的论证只允许我们断定 $\neg\neg A$, 并且读者可以验证的是,没有明显的方法从 $\neg\neg A$ (即从 $(A \rightarrow \mathbf{0}) \rightarrow \mathbf{0}$) 得到 A 。

上述将逻辑连接词翻译为类型形成操作被称为 **命题即类型**: 它为我们提供了一种将用英语书写的命题及其证明翻译为类型及其元素的方法。例如,假设我们想证明以下重言式(“德摩根律”之一):

$$\text{“若非 } A \text{ 且非 } B, \text{ 则非}(A \text{ 或 } B)\text{”}. \quad (1.11.1)$$

这个事实的一个普通英语证明可能如下进行。

⁹更准确地说,没有基本方法证明一个矛盾,即 **0** 没有构造子。如果我们的类型论不一致,那么会有某种更复杂的方法来构造 **0** 的一个元素。

假设非 A 且非 B , 并且也假设 A 或 B ; 我们将推导出一个矛盾。有两种情况。如果 A 成立, 那么由于非 A , 我们有一个矛盾。类似地, 如果 B 成立, 那么由于非 B , 我们也有一个矛盾。因此我们在两种情况下都有一个矛盾, 所以非(A 或 B)。

现在, 根据上面给出的规则, 对应我们的重言式 (1.11.1) 的类型是

$$(A \rightarrow 0) \times (B \rightarrow 0) \rightarrow (A + B \rightarrow 0) \quad (1.11.2)$$

因此我们应该能够将上述证明翻译成这个类型的一个元素。

作为这种翻译如何工作的一个例子, 让我们描述一个阅读上述英语证明的数学家如何同时在脑海中构造 (1.11.2) 的一个元素。引导短语“假设非 A 且非 B ”翻译为定义一个函数, 隐式地应用了其定义域 $(A \rightarrow 0) \times (B \rightarrow 0)$ 上 Cartesian 积的递归原理。这引入了(假设)类型为 $A \rightarrow 0$ 和 $B \rightarrow 0$ 的未命名变量。在翻译成类型论时, 我们必须给这些变量命名; 让我们称它们为 x 和 y 。在这一点上, 我们对 (1.11.2) 的一个元素的部分定义可以写成

$$f((x, y)) \equiv \square : A + B \rightarrow 0$$

其中“洞” $\square$ 的类型为 $A + B \rightarrow 0$, 表示还有待完成的部分。(我们可以等价地写成 $f \equiv \text{rec}_{(A \rightarrow 0) \times (B \rightarrow 0)}(A + B \rightarrow 0, \lambda x. \lambda y. \square)$, 使用递归子而不是模式匹配。)下一个短语“并且也假设 A 或 B ; 我们将推导出一个矛盾”表示通过一个函数定义来填充这个洞, 引入另一个未命名假设 $z : A + B$, 导致证明状态:

$$f((x, y))(z) \equiv \square : 0.$$

现在说“有两种情况”表示情况分析, 即应用余积 $A + B$ 的递归原理。如果我们使用递归子来写这个, 它将是

$$f((x, y))(z) \equiv \text{rec}_{A+B}(0, \lambda a. \square, \lambda b. \square, z)$$

而如果我们使用模式匹配来写它, 它将是

$$\begin{aligned} f((x, y))(\text{inl}(a)) &\equiv \square : 0 \\ f((x, y))(\text{inr}(b)) &\equiv \square : 0. \end{aligned}$$

注意在两种情况下, 我们现在有两个类型为 0 的“洞”需要填充, 对应于我们必须推导出矛盾的两种情况。最后, 从 $a : A$ 和 $x : A \rightarrow 0$ 得出矛盾的结论只是将函数 x 应用于 a , 另一种情况类似。(注意“应用一个函数”与“应用一个假设”或定理的短语的方便巧合。)因此我们最终的定义是

$$\begin{aligned} f((x, y))(\text{inl}(a)) &\equiv x(a) \\ f((x, y))(\text{inr}(b)) &\equiv y(b). \end{aligned}$$

对于任何类型 A 和 B , 使用我们刚刚引入的规则, 可以通过展示

$$((A + B) \rightarrow 0) \rightarrow (A \rightarrow 0) \times (B \rightarrow 0),$$

来验证逆重言式“若非(A 或 B), 则 (非 A) 且 (非 B)”。我们将此作为一个练习。

然而, 并非所有经典重言式在这种解释下都成立。例如, 规则“若非(A 且 B), 则 (非 A) 或 (非 B)”是无效的: 我们通常不能构造对应类型

$$((A \times B) \rightarrow 0) \rightarrow (A \rightarrow 0) + (B \rightarrow 0).$$

的一个元素。这反映了类型论的“自然”命题即类型逻辑是构造性的。这意味着它不包括某些经典原理, 例如排中律 (LEM) 或反证法, 以及依赖于它们的其他原理, 例如德摩根律的这个实例。

从哲学上讲, 构造性逻辑之所以如此命名, 是因为它将自身限制在可以有效地执行的构造, 也就是说那些具有计算意义的构造。在不过于精确的情况下, 这意味着有某种算法逐步指定如何构建一个对象(并且, 作为一种特殊情况, 如何看到定理为真)。这需要省略 LEM, 因为没有有效的程序来决定一个命题是真还是假。

类型论逻辑的构造性意味着它具有内在的计算意义, 这对计算机科学家来说是有趣的。这也意味着类型论提供了公理自由。例如, 虽然默认情况下没有构造见证 LEM, 但逻辑仍然与它的存在兼容(见第3.4节)。因此, 因为类型论并不否认 LEM, 我们可以一致地将其作为假设添加, 并无限制地进行常规工作。在这方面, 类型论丰富而非约束了常规的数学实践。

我们鼓励不熟悉构造性逻辑的读者通过更多例子来熟悉它。参见练习1.12和练习1.13获取一些建议。

到目前为止, 我们只讨论了命题逻辑。现在我们考虑谓词逻辑, 其中除了像“且”和“或”这样的逻辑连接词之外, 我们还有量词“存在”和“任意”。在这种情况下, 类型扮演双重角色: 它们既作为命题, 也作为传统意义上的类型, 即我们量化的论域。在类型 A 上的谓词被表示为一个族 $P : A \rightarrow \mathcal{U}$, 为每个元素 $a : A$ 分配一个类型 $P(a)$ 对应 P 对 a 成立的命题。我们现在扩展上述翻译, 解释量词:

英语	类型论
对所有 $x : A$, $P(x)$ 成立	$\prod_{(x:A)} P(x)$
存在 $x : A$ 使得 $P(x)$	$\sum_{(x:A)} P(x)$

如前所述, 我们可以证明(构造性)谓词逻辑的重言式翻译为被居留的类型。例如, 若对所有 $x : A$, $P(x)$ 且 $Q(x)$ 则 (对所有 $x : A$, $P(x)$) 且 (对所有 $x : A$, $Q(x)$) 翻译为

$$(\prod_{(x:A)} P(x) \times Q(x)) \rightarrow (\prod_{(x:A)} P(x)) \times (\prod_{(x:A)} Q(x)).$$

这个重言式的一个非正式证明可能如下进行:

假设对所有 x , $P(x)$ 且 $Q(x)$ 。首先, 我们假设给定 x 并证明 $P(x)$ 。由假设, 我们有 $P(x)$ 且 $Q(x)$, 因此我们有 $P(x)$ 。其次, 我们假设给定 x 并证明 $Q(x)$ 。再次由假设, 我们有 $P(x)$ 且 $Q(x)$, 因此我们有 $Q(x)$ 。

第一句开始通过引入其假设的一个见证来将一个蕴涵定义为一个函数:

$$f(p) := \square : (\prod_{(x:A)} P(x)) \times (\prod_{(x:A)} Q(x)).$$

在这一点上, 隐式使用了配对构造子来产生一个积类型的元素, 在这个例子中通过词语“首先”和“其次”有所指示:

$$f(p) := \left(\square : \prod_{(x:A)} P(x), \square : \prod_{(x:A)} Q(x) \right).$$

短语“我们假设给定 x 并证明 $P(x)$ ”现在表示以通常方式定义一个依值函数, 为其输入引入一个变量。由于这是在配对构造子内部, 自然地将其写为 λ -抽象:

$$f(p) := \left(\lambda x. (\square : P(x)), \square : \prod_{(x:A)} Q(x) \right).$$

现在“我们有 $P(x)$ 且 $Q(x)$ ”调用假设, 获得 $p(x) : P(x) \times Q(x)$, 而“因此我们有 $P(x)$ ”隐式应用了适当的投影:

$$f(p) := \left(\lambda x. \text{pr}_1(p(x)), \square : \prod_{(x:A)} Q(x) \right).$$

接下来两句以明显的方式填充另一个空缺:

$$f(p) := \left(\lambda x. \text{pr}_1(p(x)), \lambda x. \text{pr}_2(p(x)) \right).$$

当然, 我们用作例子的英语证明比数学家在实践中通常使用的要冗长得多; 它们更像是在“证明入门”课程中使用的语言。在实践中, 数学懂得如何填补空白, 因此我们可以省略大量细节, 并且我们通常会这样做。然而, 证明的有效性标准始终是它们可以被翻译回对应类型的元素的构造。

作为一个更具体的例子, 考虑如何定义自然数的不等式。一个自然的定义是, $n \leq m$ 当存在一个 $k : \mathbb{N}$ 使得 $n + k = m$ 。(这再次使用了我们将在下一节介绍的恒等类型, 但我们不需要太多关于它们的知识。) 在命题即类型的翻译下, 这将产生:

$$(n \leq m) := \sum_{k:\mathbb{N}} (n + k = m).$$

邀请读者从这个定义证明 $\leq$ 的熟悉性质。对于严格不等式, 有几个自然的选择, 例如

$$(n < m) := \sum_{k:\mathbb{N}} (n + \text{succ}(k) = m)$$

或

$$(n < m) := (n \leq m) \times \neg(n = m).$$

前者在构造性数学中更自然, 但在这种情况下它实际上等价于后者, 因为 $\mathbb{N}$ 具有“可判定的相等性”(见第3.4节和命题7.2.6)。

类型 $\sum_{(x:A)} P(x)$ 也有另一种解释。由于它的一个居留元是一个元素 $x : A$ 连同 $P(x)$ 成立的一个见证, 我们可以将 $\sum_{(x:A)} P(x)$ 视为“所有满足 $P(x)$ 的元素 $x : A$ 的类型”, 即作为 A 的一个“子类型”, 而不是将其视为命题“存在 $x : A$ 使得 $P(x)$ ”。

我们将在第3.5节回到这种解释。现在, 我们注意到它允许我们将公理纳入我们在第1.6节讨论的作为数学结构的类型的定义中。例如, 假设我们想定义一个半群为一个类型 A 配备一个二元运算 $m : A \rightarrow A \rightarrow A$ (即一个原群) 并且使得对所有 $x, y, z : A$ 我们有 $m(x, m(y, z)) = m(m(x, y), z)$ 。后一个命题由类型

$$\prod_{x,y,z:A} m(x, m(y, z)) = m(m(x, y), z),$$

表示, 所以半群的类型是

$$\text{Semigroup} := \sum_{(A:\mathcal{U})} \sum_{(m:A \rightarrow A \rightarrow A)} \prod_{(x,y,z:A)} m(x, m(y, z)) = m(m(x, y), z),$$

即由半群组成的 Magma 的子类型。从 Semigroup 的一个居留元中, 我们可以通过应用适当的投影来提取载体 A 、运算 m 和公理的一个见证。我们将在第2.14节回到这个例子。

还要注意, 我们可以使用类型论中的宇宙来表示“高阶逻辑”——即我们可以量化所有命题或所有谓词。例如, 我们可以将命题对所有性质 $P : A \rightarrow \mathcal{U}$, 若 $P(a)$ 则 $P(b)$ 表示为

$$\prod_{P:A \rightarrow \mathcal{U}} P(a) \rightarrow P(b)$$

其中 $A : \mathcal{U}$ 且 $a, b : A$ 。然而, 先验地, 这个命题存在于与我们正在量化的命题不同的、更高的宇宙中; 即

$$\left(\prod_{P:A \rightarrow \mathcal{U}_i} P(a) \rightarrow P(b) \right) : \mathcal{U}_{i+1}.$$

我们将在第3.5节回到这个问题。

我们在这里描述了一种“证明相关”的命题翻译, 其中析取和存在陈述的证明携带一些信息。例如, 如果我们有 $A + B$ 的一个居留元, 被视为“ A 或 B ”的一个见证, 那么我们知道它是来自 A 还是来自 B 。类似地, 如果我们有 $\sum_{(x:A)} P(x)$ 的一个居留元, 被视为“存在 $x : A$ 使得 $P(x)$ ”的一个见证, 那么我们知道元素 x 是什么(它是给定居留元的第一投影)。

作为这种逻辑的证明相关性质的一个结果, 我们可能有“ A 当且仅当 B ”(回想一下, 这意味着 $(A \rightarrow B) \times (B \rightarrow A)$), 然而类型 A 和 B 表现出不同的行为。例如, 很容易验证“ $\mathbb{N}$ 当且仅当 $\mathbf{1}$ ”, 然而显然 $\mathbb{N}$ 和 $\mathbf{1}$ 在重要方面不同。陈述“ $\mathbb{N}$ 当且仅当 $\mathbf{1}$ ”仅告诉我们, 当被视为单纯命题时, 类型 $\mathbb{N}$ 表示与 $\mathbf{1}$ 相同的命题(在这种情况下, 真命题)。我们有时通过说 A 和 B 是**逻辑等价的**来表达“ A 当且仅当 B ”。这要与在第2.4章和第4章引入的更强的类型等价概念区分开来: 尽管 $\mathbb{N}$ 和 $\mathbf{1}$ 是逻辑等价的, 但它们不是等价的类型。

在第3章, 我们将引入一类称为“单纯命题”的类型, 对于它们, 等价和逻辑等价是一致的。使用这些类型, 我们将引入对上述逻辑的一种修改, 这种修改有时是合适的, 其中析取和存在量词中包含的额外信息被丢弃。

最后, 我们注意到命题即类型的对应关系可以反向看待, 允许我们将任何类型 A 视为一个命题, 我们通过展示 A 的一个元素来证明它。有时我们将这个命题陈述为“ A 是**被居留的**”。也就是说, 当我们说 A 被居留时, 我们的意思是我们已经给出了 A 的一个(特定)元素, 但我们选择不给予该元素一个名称。类似地, 说 A 不被居留等同于给出 $\neg A$ 的一个元素。特别地, 空类型 $\mathbf{0}$ 显然不被居留, 因为 $\neg \mathbf{0} \equiv (\mathbf{0} \rightarrow \mathbf{0})$ 被 id_0 居留。¹⁰

1.12 恒等类型

虽然之前的构造可以视为标准集合论构造的推广, 但我们处理相等性的方式似乎是类型论特有的。根据命题即类型的构想, 两个相同类型的元素 $a, b : A$ 相等的命题必须对应某个类型。由于这个命题依赖于 a 和 b 是什么, 这些**相等类型**或**恒等类型**必须是依赖于 A 的两个副本的类型族。

我们可以将该类型族写作 $\text{Id}_A : A \rightarrow A \rightarrow \mathcal{U}$ (不要与恒等函数 id_A 混淆), 使得 $\text{Id}_A(a, b)$ 是表示 a 和 b 之间相等命题的类型。然而, 一旦我们熟悉了命题即类型的思想, 为了方便起见, 也使用标准的等号表示这一类型; 因此“ $a = b$ ”也将是类型 $\text{Id}_A(a, b)$ 的记法, 对应 a 等于 b 的命题。为清晰起见, 我们也可以写作“ $a =_A b$ ”来指定类型 A 。如果我们有 $a =_A b$ 的一个元素, 我们可以说 a 和 b 是**相等的**, 或者有时说**命题性相等的**, 以强调这与第1.1节讨论的判断相等 $a \equiv b$ 不同。

正如我们在第1.11节中提到的, “或”和“存在”的命题即类型版本可以包含比命题为真这一事实更多的信息, 没有什么能阻止类型 $a = b$ 也包含更多信息。实际上, 这是同伦诠释的基石, 我们将 $a = b$ 的见证视为空间 A 中 a 和 b 之间的道路或等价。正如一个空间中两点之间可以有多条道路, 两个对象相等的见证也可以不止一个。换句话说, 我们可以将 $a = b$ 视为 a 和 b 的等同化类型, 并且 a 和 b 可以以多种不同方式被等同化。我们将在第2章回到这个诠释; 现在我们将专注于恒等类型的基本规则。就像本章中考虑的所有其他类型一样, 它将有形成、引入、消去和计算的规则, 这些规则在形式上表现完全相同。

形成规则说, 给定一个类型 $A : \mathcal{U}$ 和两个元素 $a, b : A$, 我们可以在同一宇宙中形成类型 $(a =_A b) : \mathcal{U}$ 。构造 $a = b$ 元素的基本方式是知道 a 和 b 是相同的。因此, 引入规则是一个依值函数

$$\text{refl} : \prod_{a:A} (a =_A a)$$

称为**自反性**, 它说 A 的每个元素都等于它自己(以指定的方式)。我们将 refl_a 视为点 a 处的常道路。

特别地, 这意味着如果 a 和 b 是判断相等的, $a \equiv b$, 那么我们也有一个元素 $\text{refl}_a : a =_A b$ 。这是良类型的, 因为 $a \equiv b$ 意味着类型 $a =_A b$ 也判断相等于 $a =_A a$, 而后者是 refl_a 的类型。

¹⁰这不应与类型论是一致的陈述混淆, 后者是元理论的声称, 即不可能通过遵循类型论的规则获得 $\mathbf{0}$ 的一个元素。

恒等类型的归纳原理(即消去规则)是类型论中最微妙的部分之一, 并且对同伦诠释至关重要。我们首先考虑它的一个重要推论, 即“相等者可以相互替换”的原则, 如下所述:

同一者的不可分性: 对于每个族

$$C : A \rightarrow \mathcal{U}$$

存在一个函数

$$f : \prod_{(x,y:A)} \prod_{(p:x=Ay)} C(x) \rightarrow C(y)$$

使得

$$f(x, x, \text{refl}_x) \equiv \text{id}_{C(x)}.$$

这表示每个类型族 C 都尊重相等性, 意思是将 C 应用于 A 的相等元素也会在结果类型之间产生一个函数。所显示的等式表明与自反性关联的函数是恒等函数(并且我们将看到, 一般来说, 函数 $f(x, y, p) : C(x) \rightarrow C(y)$ 总是类型的一个等价)。

同一者的不可分性可以视为恒等类型的递归原理, 类似于上面为布尔值和自然数给出的递归原理。正如 $\text{rec}_{\mathbb{N}}$ 为任何其他某种类型 C 给出一个指定的映射 $\mathbb{N} \rightarrow C$, 同一者的不可分性给出一个从 $x =_A y$ 到 A 上某些其他自反二元关系的指定映射, 即对于某个一元谓词 $C(x)$ 形如 $C(x) \rightarrow C(y)$ 的那些关系。我们也可以针对更一般形式 $C(x, y)$ 的自反关系制定一个更一般的递归原理。然而, 为了完全刻画恒等类型, 我们必须将这个递归原理推广为归纳原理, 它不仅考虑从 $x =_A y$ 出发的映射, 而且考虑其上的族。换句话说, 我们不仅考虑允许用相等者替换相等者, 而且考虑相等性的证据 p 。

1.12.1 道路归纳

恒等类型的归纳原理称为**道路归纳**, 考虑到将在第2章引言中解释的同伦诠释。它可以被视为这样一个陈述: 恒等类型族是由形式为 $\text{refl}_x : x = x$ 的元素自由生成的。

道路归纳: 给定一个族

$$C : \prod_{x,y:A} (x =_A y) \rightarrow \mathcal{U}$$

和一个函数

$$c : \prod_{x:A} C(x, x, \text{refl}_x),$$

存在一个函数

$$f : \prod_{(x,y:A)} \prod_{(p:x=Ay)} C(x, y, p)$$

使得

$$f(x, x, \text{refl}_x) \equiv c(x).$$

注意, 就像乘积、余积、自然数等等的归纳原理一样, 道路归纳允许我们定义指定的函数, 这些函数展现出适当的计算行为。正如我们有通过从 $c_0 : C$ 和 $c_s : \mathbb{N} \rightarrow C \rightarrow C$ 递归定义的这样一个函数 $f : \mathbb{N} \rightarrow C$, 并且它满足 $f(0) \equiv c_0$ 和 $f(\text{succ}(n)) \equiv c_s(n, f(n))$, 我们有这样一个函数 $f : \prod_{(x,y:A)} \prod_{(p:x=Ay)} C(x, y, p)$ 从 $c : \prod_{(x:A)} C(x, x, \text{refl}_x)$ 通过道路归纳定义, 其进一步满足 $f(x, x, \text{refl}_x) \equiv c(x)$ 。

为了理解这个原理的含义, 首先考虑 C 不依赖于 p 的更简单情况。那么我们有 $C : A \rightarrow A \rightarrow \mathcal{U}$, 我们可以将其视为依赖于 A 的两个元素的谓词。我们感兴趣的是知道命题 $C(x, y)$ 何时对某些元素对 $x, y : A$ 成立。在这种情况下, 道路归纳的假设说我们知道 $C(x, x)$ 对所有 $x : A$ 成立, 即如

果我们在对 x, x 处计算 C , 我们得到一个真命题 — 所以 C 是一个自反关系。然后结论告诉我们, 只要 $x = y$, $C(x, y)$ 就成立。这正是上面提到的更一般的自反关系递归原理。

一般的归纳形式的规则允许 C 也依赖于 x 和 y 之间等同性的见证 $p : x = y$ 。在前提中, 我们不仅将 x, y 替换为 x, x , 而且同时将 p 替换为自反性: 要证明对所有元素 x, y 和它们之间的道路 $p : x = y$ 的某个性质, 只需考虑所有元素是 x, x 且道路是 $\text{refl}_x : x = x$ 的情况。如果我们只将类型视为集合, 不清楚这能给我们带来什么, 但由于可能有许多不同的等同化 $p : x = y$ 在 x 和 y 之间, 在考虑类型 $x =_A y$ 上的族时跟踪它们是有意义的。在第2章中, 我们将看到这对同伦诠释非常重要。

如果我们将道路归纳打包成一个单一函数, 它采取以下形式:

$$\text{ind}_{=_A} : \prod_{(C : \prod_{(x, y : A)} (x =_A y) \rightarrow \mathcal{U})} \left(\prod_{(x : A)} C(x, x, \text{refl}_x) \right) \rightarrow \prod_{(x, y : A)} \prod_{(p : x =_A y)} C(x, y, p)$$

具有等式

$$\text{ind}_{=_A}(C, c, x, x, \text{refl}_x) \equiv c(x).$$

函数 $\text{ind}_{=_A}$ 传统上称为 J 。我们将在引理2.3.1中展示同一者的不可性是道路归纳的一个实例, 并给它一个新的名称和记号。

给定一个证明 $p : a = b$, 道路归纳要求我们将两者 a 和 b 替换为相同的未知元素 x ; 因此为了定义族 C 的一个元素, 对于 A 的所有相等元素对, 只需在对角线上定义它。然而, 在一些证明中, 通过将所有出现的 b 替换为 a (或反之) 来利用等式 $p : a = b$ 更简单, 因为对于等式中提到的特定元素 a 完成证明的剩余部分有时比对于一般未知的 x 更容易。这激发了恒等类型的第二个归纳原理, 它说类型族 $a =_A x$ 是由元素 $\text{refl}_a : a = a$ 生成的。如下所示, 第二个原理等价于第一个; 它只是有时更方便的表述。

基点道路归纳: 固定一个元素 $a : A$, 并假设给定一个族

$$C : \prod_{x : A} (a =_A x) \rightarrow \mathcal{U}$$

和一个元素

$$c : C(a, \text{refl}_a).$$

那么我们得到一个函数

$$f : \prod_{(x : A)} \prod_{(p : a =_A x)} C(x, p)$$

使得

$$f(a, \text{refl}_a) \equiv c.$$

此处, $C(x, p)$ 是一个类型族, 其中 x 是 A 的一个元素, p 是恒等类型 $a =_A x$ 的一个元素, 对于 A 中固定的 a 。基点道路归纳原理说的是, 要为所有 x 和 p 定义这个族的一个元素, 只需考虑 x 是 a 且 p 是 $\text{refl}_a : a = a$ 的情况。

打包为一个函数, 基点道路归纳变为:

$$\text{ind}'_{=_A} : \prod_{(a : A)} \prod_{(C : \prod_{(x : A)} (a =_A x) \rightarrow \mathcal{U})} C(a, \text{refl}_a) \rightarrow \prod_{(x : A)} \prod_{(p : a =_A x)} C(x, p)$$

具有等式

$$\text{ind}'_{=_A}(a, C, c, a, \text{refl}_a) \equiv c.$$

下面, 我们展示道路归纳和基点道路归纳是等价的。因此, 我们有时会马虎地也将基点道路归纳简称为“道路归纳”, 依靠读者从证明的形式推断出是哪个原理。

Remark 1.12.1. 直观上, 自然数的归纳原理表达了每个自然数要么是 0 要么是某个自然数 n 的形式 $\text{succ}(n)$ 这一事实, 因此如果我们在这些情况下证明一个性质(在第二种情况下使用归纳假设), 那么我们就为所有自然数证明了它。类似地, $A + B$ 的归纳原理表达了 $A + B$ 的每个元素要么是形式 $\text{inl}(a)$ 要么是 $\text{inr}(b)$ 等等这一事实。将这种读法应用于道路归纳, 我们可能会说道路归纳表达了每条道路都是形式 refl_a 这一事实, 因此如果我们在自反性道路上证明一个性质, 那么我们就为所有道路证明了它。

然而, 在道路的同伦诠释背景下, 这种读法相当令人困惑, 因为两个元素 a 和 b 可以以许多不同方式被等同化, 因此恒等类型可能有许多不同的元素! 怎么可能有许多不同的道路, 但同时我们有一个归纳原理断言唯一的道路是自反性?

关键在于, 要观察到通过归纳定义的不是恒等类型, 而是恒等族。特别地, 道路归纳说, 类型族 $(x =_A y)$, 当 x, y 在 A 的所有元素上变化时, 是由形式为 refl_x 的元素归纳定义的。这意味着要为依赖于恒等族的一般元素 (x, y, p) 的任何其他族 $C(x, y, p)$ 给出一个元素, 只需考虑形式为 (x, x, refl_x) 的情况。在同伦诠释中, 这说三元组 (x, y, p) 的类型, 其中 x 和 y 是道路 p 的端点(换句话说, $\Sigma_{(x, y: A)}(x = y)$), 是由每个点 x 处的常环路归纳生成的。正如我们将在第2章中看到的, 在同伦论中对应于 $\Sigma_{(x, y: A)}(x = y)$ 的空间是自由道路空间 — A 中端点可以变化的道路的空间 — 并且事实上这个空间的任何点都同伦于某个点处的常环路, 因为我们可以简单地沿着给定的道路收缩它的一个端点。类似的事实也在类型论中成立: 我们可以通过对 $p : x = y$ 的道路归纳证明 $(x, y, p) =_{\Sigma_{(x, y: A)}(x = y)} (x, x, \text{refl}_x)$ 。

类似地, 基点道路归纳说, 对于固定的 $a : A$, 类型族 $(a =_A y)$, 当 y 在 A 的所有元素上变化时, 是由元素 refl_a 归纳定义的。因此, 要为依赖于这个族的一般元素 (y, p) 的任何其他族 $C(y, p)$ 给出一个元素, 只需考虑情况 (a, refl_a) 。在同伦上, 这表达了从某个选定点出发的道路空间(该点处的基于道路的空间, 类型论上是 $\Sigma_{(y: A)}(a = y)$)可缩到选定点上的常环路这一事实。同样, 相应的事实也在类型论中成立: 我们可以通过对 $p : a = y$ 的基点道路归纳证明 $(y, p) =_{\Sigma_{(y: A)}(a = y)} (a, \text{refl}_a)$ 。还要注意, 根据在第1.11节提到的将 Σ -类型解释为子类型, 类型 $\Sigma_{(y: A)}(a = y)$ 可以被视为“所有等于 a 的 A 元素的类型”, 这是“独元子集” $\{a\}$ 的类型论版本。

道路归纳和基点道路归纳都没有提供一种方法来给出族 $C(p)$ 的一个元素, 其中 p 有两个固定端点 a 和 b 。特别地, 对于依赖于一个环路的族 $C : (a =_A a) \rightarrow \mathcal{U}$, 我们不能应用道路归纳而只考虑 $C(\text{refl}_a)$ 的情况, 因此, 我们不能证明所有环路都是自反性。因此, 归纳定义恒等族并不禁止恒等类型的特定实例中的非自反性道路。换句话说, 一条道路 $p : x = x$ 作为 $(x = x)$ 的一个元素可能不等于自反性, 但对 (x, p) 将仍然等于对 (x, refl_x) 作为 $\Sigma_{(y: A)}(x = y)$ 的元素。

作为一个拓扑例子, 考虑在穿孔圆盘 $\{(x, y) \mid 0 < x^2 + y^2 < 2\}$ 中的一个环路, 它从 $(1, 0)$ 开始, 绕孔 $(0, 0)$ 一圈后返回到 $(1, 0)$ 。如果我们将两个端点都固定在 $(1, 0)$, 这个环路不能在穿孔圆盘内形变为一条常道路, 就像绕在杆子上的绳子圈如果我们握住两端就不能拉进来一样。然而, 如果我们允许一个端点变化, 环路可以收缩回常道路, 就像如果我们只握住一端, 我们总是可以收进绳子一样。

1.12.2 道路归纳与基点道路归纳的等价性

上面介绍的恒等类型的两个归纳原理是等价的。很容易看出道路归纳可以从基点道路归纳原理推导出来。确实, 让我们假设道路归纳的前提:

$$\begin{aligned} C &: \prod_{x, y: A} (x =_A y) \rightarrow \mathcal{U}, \\ c &: \prod_{x: A} C(x, x, \text{refl}_x). \end{aligned}$$

现在, 给定一个元素 $x : A$, 我们可以实例化上述两者, 得到

$$\begin{aligned} C' &: \prod_{y:A} (x =_A y) \rightarrow \mathcal{U}, \\ C' &:\equiv C(x), \\ c' &: C'(x, \text{refl}_x), \\ c' &:\equiv c(x). \end{aligned}$$

显然, C' 和 c' 匹配基点道路归纳的前提, 因此我们可以构造

$$g : \prod_{(y:A)} \prod_{(p:x=y)} C'(y, p)$$

具有定义等式

$$g(x, \text{refl}_x) :\equiv c'.$$

现在我们观察到 g 的陪域等于 $C(x, y, p)$ 。因此, 解除我们对 $x : A$ 的假设, 我们可以推导出一个函数

$$f : \prod_{(x,y:A)} \prod_{(p:x=Ay)} C(x, y, p)$$

具有所需的判断等式 $f(x, x, \text{refl}_x) \equiv g(x, \text{refl}_x) :\equiv c' :\equiv c(x)$ 。

这个事实的另一个证明是观察到任何这样的 f 都可以作为 $\text{ind}_{=A}$ 的一个实例获得, 因此只需用 $\text{ind}'_{=A}$ 定义 $\text{ind}_{=A}$ 为

$$\text{ind}_{=A}(C, c, x, y, p) :\equiv \text{ind}'_{=A}(x, C(x), c(x), y, p).$$

另一个方向有点棘手; 我们不清楚如何使用道路归纳的一个特定实例来推导基点道路归纳的一个特定实例。相反, 我们可以做的是构造一个道路归纳的实例, 它一次展示所有可能的基点道路归纳的实例化。定义

$$\begin{aligned} D &: \prod_{x,y:A} (x =_A y) \rightarrow \mathcal{U}, \\ D(x, y, p) &:\equiv \prod_{C:\prod_{(z:A)} (x =_A z) \rightarrow \mathcal{U}} C(x, \text{refl}_x) \rightarrow C(y, p). \end{aligned}$$

然后我们可以构造函数

$$\begin{aligned} d &: \prod_{x:A} D(x, x, \text{refl}_x), \\ d &:\equiv \lambda x. \lambda C. \lambda (c : C(x, \text{refl}_x)). c \end{aligned}$$

因此使用道路归纳得到

$$f : \prod_{(x,y:A)} \prod_{(p:x=Ay)} D(x, y, p)$$

其中 $f(x, x, \text{refl}_x) :\equiv d(x)$ 。展开 D 的定义, 我们可以扩展 f 的类型:

$$f : \prod_{(x,y:A)} \prod_{(p:x=Ay)} \prod_{(C:\prod_{(z:A)} (x =_A z) \rightarrow \mathcal{U})} C(x, \text{refl}_x) \rightarrow C(y, p).$$

现在给定 $a : A$ 连同 $x : A$ 和 $p : a =_A x$, 我们可以推导出基点道路归纳的结论:

$$f(a, x, p, C, c) : C(x, p).$$

注意我们也得到了正确的定义等式。

另一个证明是观察到任何基点道路归纳的使用都是 $\text{ind}'_{=A}$ 的一个实例, 并定义

$$\text{ind}'_{=A}(a, C, c, x, p) \equiv \text{ind}_{=A}((\lambda x, y. \lambda p. \prod_{(C: \prod_{(z:A)} (x =_A z) \rightarrow \mathcal{U})} C(x, \text{refl}_x) \rightarrow C(y, p)), \\ (\lambda x. \lambda C. \lambda d. d), a, x, p)(C, c).$$

注意上面给出的构造使用了宇宙。也就是说, 如果我们想用 $C : \prod_{(x:A)} (a =_A x) \rightarrow \mathcal{U}_i$ 来建模 $\text{ind}'_{=A}$, 我们需要使用 $\text{ind}_{=A}$ 与

$$D : \prod_{x, y: A} (x =_A y) \rightarrow \mathcal{U}_{i+1}$$

因为 D 量化了所有给定类型的 C 。虽然这与我们对宇宙的定义兼容, 但也可以在不使用宇宙的情况下推导 $\text{ind}'_{=A}$: 我们可以证明 $\text{ind}_{=A}$ 蕴含引理2.3.1和定理3.11.8, 并且这两个原理直接蕴含 $\text{ind}'_{=A}$ 。我们将细节留给读者作为练习1.7。

我们可以使用上述任一恒等类型的公式化来建立相等性是一个等价关系, 每个函数都保持相等性, 并且每个族都尊重相等性。我们将细节留给下一章, 在那里这将在同伦类型论的背景下被推导和解释。

Remark 1.12.2. 我们强调, 尽管具有一些不熟悉的特征, 命题相等是同伦类型论中数学的那个相等性。这种区分不属于判断相等, 判断相等更像是类型论规则的元理论特征。例如, 在第1.9节中证明的自然数加法的结合律是一个命题相等, 而不是判断相等。交换律也是如此(练习1.16)。甚至非常简单的交换律 $n + 1 = 1 + n$ 对于一般的 n 也不是判断相等(尽管对于任何特定的 n 它是判断的, 例如 $3 + 1 \equiv 1 + 3$, 因为根据定义 $+$ 的计算规则, 两者都判断相等于 4)。我们只能通过使用恒等类型来证明这样的事实, 因为我们只能将 $\mathbb{N}$ 的归纳原理应用于一个类型作为输出(而不是一个判断)。

1.12.3 不相等性

最后, 让我们也谈谈不相等性, 它是相等性的否定:¹¹

$$(x \neq_A y) \equiv \neg(x =_A y).$$

如果 $x \neq y$, 我们说 x 和 y 是**不相等的** 或**不相等**。就像否定一样, 不相等性在这里的作用不如在经典数学中那么重要。例如, 我们不能通过证明两个东西不是不相等来证明它们相等: 那将是经典双重否定律的应用, 见第3.4节。

有时以积极的方式表述不相等性是有用的。例如, 在定理 11.2.4 中, 我们将证明一个实数 x 有逆当且仅当它与 0 的距离是正的, 这是一个比 $x \neq 0$ 更强的要求。

备注

这里介绍的类型论是 Martin-Löf 的直觉主义类型论 [ML98, ML75, ML82, ML84] 的一个版本, 它本身基于并受到 Brouwer [Bee85], Heyting [Hey66], Scott [Sco70], de Bruijn [dB73], Howard [How80], Tait [Tai67, Tai68], 以及 Lawvere [Law06] 的基础性工作的影响。Martin-Löf 类型论的三个主要变体构成了 NuPRL [CAB⁺86], Coq [Coq12], 和 AGDA [Nor07] 这些类型论计算机实现的基础。这里给出的理论与这些形式化在若干方面有所不同, 其中一些对同伦诠释至关重要, 而其他一些则是技术上的便利或涉及尚未在同伦设定下研究的概念。

最重要的是, 这里描述的类型论源自 Martin-Löf 类型论的内涵版本 [ML75], 而非外延版本 [ML82]。外延理论不区分判断相等和命题相等, 而内涵理论将判断相等视为纯粹定义性的, 并允许

¹¹我们使用“不等式”来指 $<$ 和 $\leq$ 。另外, 注意这是命题恒等类型的否定。当然, 否定判断相等 $\equiv$ 没有意义, 因为判断不受逻辑操作影响。

对恒等类型进行更广泛, 证明相关的解释, 这对同伦诠释至关重要. 从同伦视角看, 外延类型论将自身局限于同伦离散集(见§3.1), 而内涵理论则允许具有更高维结构的类型. NuPRL系统 [CAB⁺86] 是外延的, 而 Coq [Coq12] 和 AGDA [Nor07] 都是内涵的. 在内涵类型论中, 存在许多在恒等证明结构上不同的变体. 我们这里依赖的最自由的解释允许对相等性进行证明相关的解释, 而更受限制的变体则施加了诸如恒等证明唯一性 (UIP) [Str93] 之类的限制, 该性质断言任何两个相等性证明都是判断相等的, 以及公理 K [Str93], 它断言唯一的相等性证明是自反性(模判断相等). 这些额外要求可以在 Coq 和 AGDA 系统中选择性地施加.

内涵理论之间的另一个不同之处在于判断相等的强度, 特别是关于函数类型的对象. 这里我们包含唯一性原理 (η -转换) $f \equiv \lambda x. f(x)$ 作为判断相等的一条原则. 例如, 在§4.9中, 这个原理被用于证明泛等公理蕴含命题函数外延性. 唯一性原理有时也考虑用于其他类型. 例如, Cartesian 积 $A \times B$ 的唯一性原理 将是我们在§1.5中构造的命题等式 $\text{uniq}_{A \times B}$ 的判断版本, 即断言 $u \equiv (\text{pr}_1(u), \text{pr}_2(u))$. 这个以及相应的依值对版本将是合理的选择(我们并未采用), 但我们不能包含所有这样的规则, 因为恒等类型的相应唯一性原理将使所有高阶同伦结构变得平凡. 因此我们被迫将其排除, 问题实际在于在哪里划清界限. 关于归纳类型, 我们在§5.5中进一步讨论这些点.

对于我们的目的而言, 重要的是函数的(命题)相等被看作是外延的(此处的意义与上文不同!). 这并非本章规则的推论; 它将由§2.9.3表达. 这个决定对我们的目的是重要的, 因为它规定了函数的相等性符合数学中的预期. 尽管我们将§2.9.3作为一个公理包含, 但它可以从泛等公理和函数的唯一性原理(见§4.9)推导出来, 也可以从区间类型的存在性(见定理6.3.2)推导出来.

关于积, Σ -类型, 余积, 自然数等归纳类型(见第5章), 在归纳和递归的表述上存在额外的选择. 我们以归纳原理为基础, 并将模式匹配视为从中派生出来的, 但也可以反过来做; 见第A章. 在后一种情况下, 通常也允许深度模式匹配; 见 [Coq92b]. 我们做出这个选择有几个原因. 一个原因是归纳原理是我们在范畴语义中自然得到的. 另一个原因是指定允许的(深度)模式匹配种类相当棘手; 例如, AGDA 的模式匹配默认可以证明公理 K, 尽管有一个标志 `--without-K` 可以防止这一点 [CDP14]. 最后, 对于高阶归纳类型(见第6章), 深度模式匹配尚未得到很好的理解. 因此, 我们将只使用诸如§1.10中描述的那些模式匹配, 它们直接等价于归纳原理的应用.

与 Coq 的类型论不同, 我们不包含一个原始命题类型. 相反, 如§1.11中所讨论的, 我们遵循命题即类型 (PAT) 原则, 将命题与类型等同. 这最初由 de Bruijn [dB73], Howard [How80], Tait [Tai68], 和 Martin-Löf [ML98] 提出. (我们的决定在§3.2和§3.3中有更充分的解释.)

然而, 我们确实包含了一个完整的累积宇宙层次结构, 使得类型形成和相等判断成为宇宙的成员关系和相等判断的实例. 为方便起见, 我们将宇宙的对象视为类型, 而不是类型的代码; 用 [ML84] 的术语来说, 这意味着我们使用“Russell式宇宙”而非“Tarski式宇宙”. 另一种选择是使用 Tarski 式宇宙, 需要一个显式的强制转换函数将宇宙的元素 $A : \mathcal{U}$ 转换为类型 $\text{El}(A)$, 并且只说在非形式化工作时省略了强制转换.

我们还将宇宙层次结构视为累积的, 即每个在 $\mathcal{U}_i$ 中的类型也在每个 $j \geq i$ 的 $\mathcal{U}_j$ 中. 在形式化实现累积性上有不同的方法: 最简单的是只包含一条规则, 如果 $A : \mathcal{U}_i$ 则 $A : \mathcal{U}_j$. 然而, 这带来了恼人的后果, 即对于类型族 $B : A \rightarrow \mathcal{U}_i$, 我们不能得出 $B : A \rightarrow \mathcal{U}_j$, 尽管我们可以得出 $\lambda a. B(a) : A \rightarrow \mathcal{U}_j$. 一个解决这个问题的更复杂的方法是引入一个由 $\mathcal{U}_i <: \mathcal{U}_j$ 生成的判断子类型关系 $<:$, 但这使得类型论的研究更加复杂. 另一个替代方案是包含一个显式的强制转换函数 $\downarrow : \mathcal{U}_i \rightarrow \mathcal{U}_j$, 在非形式化工作时可以省略它.

宇宙也不必由自然数索引并线性排序. 对于某些目的, 更合适的假设是每个宇宙都是某个更大宇宙的元素, 再加上一个“有向性”性质, 即任何两个宇宙都共同包含在某个更大的宇宙中. 还有许多其他可能的变体, 例如包含一个包含所有 $\mathcal{U}_i$ 的宇宙“ $\mathcal{U}_\omega$ ”(甚至更高的“大基数”类型宇宙), 或者通过将层次结构内部化为一个类型族 $\lambda i. \mathcal{U}_i$. 后者实际上在 AGDA 中实现了.

恒等类型的道路归纳原理由 Martin-Löf [ML75] 阐述. Martin-Löf 类型论设定下的基点道路归纳规则归功于 Paulin-Mohring [PM93]; 它可以看作是 NuPRL [CAB⁺86, Section 8.1] 中假设判断的“逐点函数性”的内涵推广. Martin-Löf 的规则蕴含 Paulin-Mohring 的规则这一事实由 Streicher 使用公理 K 证明(见§7.2), 由 Altenkirch 和 Goguen 如§1.12中证明, 最后由 Hofmann 在没有宇宙

的情况下证明(如练习1.7); 见 [Str93, §1.3 and Addendum].

练习

Exercise 1.1. 给定函数 $f : A \rightarrow B$ 和 $g : B \rightarrow C$, 定义它们的复合 $g \circ f : A \rightarrow C$. 证明我们有 $h \circ (g \circ f) \equiv (h \circ g) \circ f$.

Exercise 1.2. 仅使用投影推导积的递归原理 $\text{rec}_{A \times B}$, 并验证定义等式是有效的. 对 Σ -类型做同样的事情.

Exercise 1.3. 仅使用投影和命题唯一性原理 $\text{uniq}_{A \times B}$ 推导积的归纳原理 $\text{ind}_{A \times B}$. 验证定义等式是有效的. 将 $\text{uniq}_{A \times B}$ 推广到 Σ -类型, 并对 Σ -类型做同样的事情. (这需要第2章中的概念.)

Exercise 1.4. 假设仅给定自然数的迭代器

$$\text{iter} : \prod_{C:\mathcal{U}} C \rightarrow (C \rightarrow C) \rightarrow \mathbb{N} \rightarrow C$$

及其定义方程

$$\begin{aligned} \text{iter}(C, c_0, c_s, 0) &\equiv c_0, \\ \text{iter}(C, c_0, c_s, \text{succ}(n)) &\equiv c_s(\text{iter}(C, c_0, c_s, n)), \end{aligned}$$

推导一个具有递归子 $\text{rec}_{\mathbb{N}}$ 类型的函数. 使用 $\mathbb{N}$ 的归纳原理证明该函数的递归子定义等式在命题意义下成立.

Exercise 1.5. 证明如果定义 $A + B \equiv \sum_{(x:\mathbb{N})} \text{rec}_2(\mathcal{U}, A, B, x)$, 那么我们可以给出 ind_{A+B} 的一个定义, 使得在§1.7中所述的定义等式成立.

Exercise 1.6. 证明如果定义 $A \times B \equiv \prod_{(x:\mathbb{N})} \text{rec}_2(\mathcal{U}, A, B, x)$, 那么我们可以给出 $\text{ind}_{A \times B}$ 的一个定义, 使得在§1.5中所述的定义等式在命题意义下成立(即使用相等类型). (这需要函数外延性公理, 该公理在§2.9中介绍.)

Exercise 1.7. 给出 $\text{ind}'_{=A}$ 从 $\text{ind}_{=A}$ 的另一种推导, 避免使用宇宙. (使用后续章节的概念会更容易.)

Exercise 1.8. 使用 $\text{rec}_{\mathbb{N}}$ 定义乘法和幂运算. 仅使用 $\text{ind}_{\mathbb{N}}$ 验证 $(\mathbb{N}, +, 0, \times, 1)$ 是一个半环. 你可能还需要使用相等的对称性和传递性, 即§2.1.1和§2.1.2.

Exercise 1.9. 定义在§1.3末尾提到的类型族 $\text{Fin} : \mathbb{N} \rightarrow \mathcal{U}$, 以及在§1.4中提到的依值函数 $\text{fmax} : \prod_{(n:\mathbb{N})} \text{Fin}(n+1)$.

Exercise 1.10. 证明 Ackermann 函数 $\text{ack} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 可以仅使用 $\text{rec}_{\mathbb{N}}$ 定义, 并满足以下方程:

$$\begin{aligned} \text{ack}(0, n) &\equiv \text{succ}(n), \\ \text{ack}(\text{succ}(m), 0) &\equiv \text{ack}(m, 1), \\ \text{ack}(\text{succ}(m), \text{succ}(n)) &\equiv \text{ack}(m, \text{ack}(\text{succ}(m), n)). \end{aligned}$$

Exercise 1.11. 证明对于任何类型 A , 我们有 $\neg\neg\neg A \rightarrow \neg A$.

Exercise 1.12. 使用命题即类型解释, 推导以下重言式.

- (i) 若 A , 则 (若 B 则 A).
- (ii) 若 A , 则非(非 A).
- (iii) 若 (非 A 或非 B), 则非(A 且 B).

Exercise 1.13. 使用命题即类型, 推导排中律的双重否定, 即证明非(非(P 或非 P)).

Exercise 1.14. 为什么恒等类型的归纳原理不允许我们构造一个函数 $f : \prod_{(x:A)} \prod_{(p:x=x)} (p = \text{refl}_x)$ 并具有定义等式

$$f(x, \text{refl}_x) :\equiv \text{refl}_{\text{refl}_x} \quad ?$$

Exercise 1.15. 证明同一者的不可分性可以从道路归纳推导出来.

Exercise 1.16. 证明自然数的加法是可交换的: $\prod_{(i,j:\mathbb{N})} (i + j = j + i)$.